

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

Гомельский государственный технический университет имени П.О.Сухого

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

специальность 6-05-0611-01 «Информационные системы и технологии»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе

по дисциплине «Распределённые информационные системы»

на тему: Семантический поиск актуальной информации о специализации
языковых моделей нейронных сетей

Исполнитель: студент группы ИТП-31

Тропец А.А.

Руководитель: доцент

Комраков В.В.

Дата проверки:

Дата допуска к защите:

Дата защиты:

Оценка работы:

Подписи членов комиссии по защите

Курсовой работы:

СОДЕРЖАНИЕ

Введение.....	3
1 Обзор источников для разработки <i>web</i> -приложения.....	4
1.1 Обзор существующих решений для проектирования приложения семантического поиска	4
1.2 Обзор архитектурных стилей создания веб-приложения	6
1.3 Программные средства для реализации информационной системы.....	9
2 Проектирование веб-приложения семантического поиска	13
2.1 Постановка задачи	13
2.2 Структура базы данных.....	15
2.3 Общая архитектура приложения	19
3 Реализация и тестирование	24
3.1 Особенности реализации приложения.....	24
3.2 Тестирование приложения	25
Заключение	30
Список использованных источников	31
Приложение А	32
Приложение Б.....	33
Приложение В.....	34
Приложение Г	40

ВВЕДЕНИЕ

Современные информационные системы характеризуются огромным ростом объёмов неструктурированных текстовых данных. Традиционные методы полнотекстового поиска, основанные на лексическом сопоставлении ключевых слов, не обеспечивают достаточной релевантности результатов, поскольку не учитывают семантическую близость понятий. В условиях распределённых информационных систем возникает потребность в механизмах интеллектуального поиска, способных понимать смысл запроса и находить информацию даже при отсутствии точного совпадения терминов.

Решением является применение технологий векторного представления текста и семантического поиска на базе нейронных сетей. Архитектурный стиль *REST* с использованием протокола *HTTP* и формата *JSON* обеспечивает независимость клиентской части от серверной реализации, прозрачность кэширования и удобство интеграции с внешними системами [1]. Многопоточная обработка запросов позволяет обеспечить высокую производительность системы при одновременном обслуживании множества пользователей.

Целью курсовой работы является разработка веб-приложения для семантического поиска информации с использованием многопоточной обработки запросов и *REST-API*.

Для достижения цели необходимо решить следующие задачи:

- проанализировать существующие программные решения для семантического поиска и архитектурные стили создания веб-приложений (*REST*, *SOAP*, *GraphQL*, *Grpc* [2]);
- обосновать выбор языка программирования, веб-фреймворка, СУБД, клиентских технологий и средств разработки;
- спроектировать архитектуру приложения и *REST-API*, определив состав ресурсов, методы *HTTP* и формат *JSON*-сообщений;
- разработать модель данных для хранения документов и их векторных представлений;
- реализовать многопоточную обработку запросов к векторному хранилищу с использованием параллельных вычислений;
- провести модульное и нагрузочное тестирование разработанного программного продукта.

Объектом разработки является веб-приложение для семантического поиска актуальной информации о специализациях языковых моделей нейронных сетей. Предметом исследования – архитектура распределённого веб-приложения и алгоритмы многопоточной обработки семантического поиска.

1 ОБЗОР ИСТОЧНИКОВ ДЛЯ РАЗРАБОТКИ WEB-ПРИЛОЖЕНИЯ

1.1 Обзор существующих решений для проектирования приложения семантического поиска

Современные распределённые информационные системы характеризуются экспоненциальным ростом объёмов данных. В области искусственного интеллекта и машинного обучения данный тренд выражен наиболее остро [3]. Постоянное появление новых архитектур нейронных сетей, методов оптимизации и инструментов разработки требует механизмов поиска, способных оперировать не лексическим совпадением терминов, а семантической близостью понятий. Традиционные полнотекстовые индексы, основанные на инвертированных списках и метриках *TF-IDF* или *BM25*, не обеспечивают релевантности результатов при перефразировании запросов или использовании синонимичных формулировок.

Актуальность темы обусловлена необходимостью систематизации знаний о специализации нейронных сетей. Предметная область развивается динамично: ежемесячно публикуются исследования, демонстрирующие преимущества трансформеров над рекуррентными сетями в задачах обработки последовательностей, диффузионных моделей над генеративно-состязательными сетями в синтезе изображений или методов эффективного обучения при ограниченных вычислительных ресурсах. Корректный выбор архитектуры зависит от специфики задачи, требований к задержкам и доступных данных. Семантический поиск позволяет находить рекомендации и технические спецификации даже при неточном формулировании запроса, что критично для инженерной и исследовательской работы.

В качестве языка взаимодействия с системой выбран английский. Данный выбор обоснован следующими факторами:

- подавляющее большинство научных публикаций, технической документации и руководств по применению нейронных сетей издаётся на английском языке [4];
- циклы разработки и выпуска новых версий моделей исчисляются неделями, что приводит к быстрому устареванию информации;
- процесс профессионального перевода технических материалов занимает значительное время, в результате чего переведённые документы теряют актуальность относительно оригинальных источников;
- использование оригинальных англоязычных формулировок исключает искажение терминологии и обеспечивает прямой доступ к первоисточникам без посредничества переводческих инструментов.

На рынке программных решений для семантического поиска представлено несколько платформ, реализующих векторное индексирование и поиск по сходству. К числу наиболее распространённых относятся *Elasticsearch*, *Pinecone*, *Weaviate*, *Qdrant* и *Meiliseach* [5]. Указанные продукты предоставляют *API* для хранения эмбедингов и выполнения запросов по косинусному сходству, однако их применение в рамках разрабатываемого проекта ограничено рядом технических и архитектурных ограничений.

Elasticsearch обеспечивает полнотекстовый и векторный поиск в едином кластере, но требует сложной настройки *Java*-окружения и потребляет значительные ресурсы памяти. *Pinecone* представляет собой полностью управляемое облачное решение, что исключает возможность локального развёртывания и контроля над сетевой задержкой. *Weaviate* и *Qdrant* ориентированы на работу с графами знаний и векторными базами данных соответственно, однако их интеграция в лёгкое веб-приложение усложняет архитектуру развёртывания. *Meiliseach* демонстрирует высокую скорость лексического поиска, но поддержка векторных операций находится на экспериментальной стадии и не обеспечивает стабильной работы с многопоточными нагрузками.

В таблице 1.1 приведён сравнительный анализ существующих программных решений по следующим критериям: поддержка векторного поиска, реализация многопоточной обработки запросов, тип лицензии, сложность интеграции и пригодность для локального развёртывания.

Таблица 1.1 – Сравнение существующих программных решений

Продукт	Векторный поиск	Многопоточность	Лицензия	Сложность интеграции	Локальное развёртывание
<i>Elasticsearch</i>	Да	Да	<i>SSPL/Elastic</i>	Высокая	Да
<i>Pinecone</i>	Да	Да	Проприетарная	Низкая	Нет
<i>Weaviate</i>	Да	Да	<i>BSD-3</i>	Средняя	Да
<i>Qdrant</i>	Да	Да	<i>Apache 2.0</i>	Средняя	Да
<i>Meiliseach</i>	Экспериментально	Да	<i>MIT</i>	Низкая	Да
Разрабатываемое решение	Да	Да	Открытая	Низкая	Да

Анализ таблицы 1.1 показывает, что коммерческие и облачные платформы либо навязывают проприетарные модели лицензирования, либо требуют выделения отдельного серверного узла, что противоречит принципу развёртывания лёгкого веб-приложения. *Open-source* решения обладают избыточной функциональностью для задач тематического поиска по базе знаний и усложняют поддержку многопоточного контура обработки запросов. Бесплатных решений с открытым исходным кодом, ориентированных на семантический и гибридный поиск в рамках клиент-серверной архитектуры с нативной поддержкой параллельных вычислений на языке C# [6], на момент выполнения работы не выявлено. Это обосновывает целесообразность разработки собственного программного продукта, реализующего семантический поиск с акцентом на многопоточную обработку векторных операций и *REST-API*.

1.2 Обзор архитектурных стилей создания веб-приложения

Под клиент-серверным веб-приложением в данной работе понимается программный комплекс, в котором интерфейс пользователя размещается в браузере, а бизнес-логика, алгоритмы ранжирования и хранилище данных разнесены на серверный узел. Ключевым архитектурным решением при построении таких систем является выбор стиля интеграции, определяющего способ обмена данными, управление состоянием сеанса и механизм масштабирования вычислительных ресурсов.

К рассмотрению выбраны четыре наиболее распространённых архитектурных стиля, применяемых при проектировании распределённых информационных систем:

- *REST (Representational State Transfer)* – стиль, основанный на ресурсной модели, где операции выполняются стандартными методами *HTTP (GET, POST, PUT, DELETE)*, а представления ресурсов передаются преимущественно в формате *JSON*;
- *GraphQL* – язык запросов к данным, позволяющий клиенту точно определять структуру возвращаемого ответа через единственный *POST*-эндпоинт;
- *gRPC* – высокопроизводительный фреймворк вызова удалённых процедур поверх *HTTP/2* с бинарной сериализацией *Protocol Buffers*;
- *WebSocket* – протокол постоянного двунаправленного соединения, ориентированный на передачу сообщений в реальном времени без повторного установления *TCP*-сессии.

В таблице 1.2 приведено сопоставление перечисленных стилей по характеристикам, значимым для системы семантического поиска с вычислительно нагруженным ядром.

Таблица 1.2 – Сравнение архитектурных стилей клиент-серверного взаимодействия

Стиль	Транспорт	Формат данных	Поддержка многопоточности	Пригодность для семантического поиска	Сложность интеграции с браузером
<i>REST</i>	<i>HTTP/1.1</i> или <i>HTTP/2</i>	<i>JSON</i>	Полная (асинхронные обработчики, пул потоков <i>ASP.NET Core</i>)	Высокая (удобная сериализация векторов и метаданных)	Низкая (нативная поддержка <i>fetch API</i>)
<i>GraphQL</i>	<i>HTTP</i>	<i>JSON</i>	Средняя (требует кастомных резолверов)	Средняя (сложность типизации бинарных эмбедингов)	Средняя (требует клиентских библиотек)
<i>gRPC</i>	<i>HTTP/2</i>	<i>Protocol Buffers</i> (бинарный)	Высокая (встроенный <i>stream-based concurrency</i>)	Низкая (отсутствие нативной поддержки в браузерах)	Высокая (требует <i>gRPC-Web</i> прокси)
<i>Web-Socket</i>	<i>TCP</i>	Произвольный (обычно <i>JSON</i>)	Средняя (управление состоянием сессий усложняет масштабирование)	Низкая (не подходит для <i>request-response</i> паттерна поиска)	Средняя

Для разрабатываемого программного продукта в качестве архитектурного стиля был выбран *REST*. Данное решение обусловлено совокупностью технических требований, предъявляемых к системе семантического поиска и многопоточной обработке запросов.

Во-первых, модель *REST* предполагает полную *stateless*-архитектуру, при которой каждый *HTTP*-запрос содержит всю необходимую информацию для его обработки. Это напрямую соответствует принципам многопоточного проектирования в *ASP.NET Core*: сервер не хранит контекст выполнения между запросами, что позволяет делегировать обработку каждого поискового запроса свободному потоку из *ThreadPool* без блокировок и гонок данных. Вычислительно сложные

операции (генерация эмбедингов через *ONNX Runtime*, параллельный расчёт косинусного сходства средствами *PLINQ*) изолируются в асинхронных задачах и выполняются независимо от транспортного уровня.

Во-вторых, формат *JSON* обеспечивает прозрачную сериализацию структур данных, используемых в алгоритме гибридного ранжирования: векторы запроса и документов, метрики косинусного сходства, веса ключевых слов и итоговые оценки релевантности. В отличие от *GraphQL*, *REST* не требует описания сложной схемы типов для числовых массивов размерностью 384 *float*, что упрощает разработку и отладку *API*. В отличие от *gRPC*, *JSON* не требует дополнительных шлюзов (*gRPC-Web*) для взаимодействия с браузерным клиентом и сохраняется в человекочитаемом виде при логировании запросов.

В-третьих, *REST*-архитектура обеспечивает естественную интеграцию с механизмами горизонтального масштабирования, что критично для распределённых информационных систем. При росте нагрузки поисковые запросы могут балансироваться между несколькими экземплярами сервера, каждый из которых независимо выполняет векторные вычисления в своём адресном пространстве. Состояние векторного хранилища синхронизируется через общую реляционную СУБД или кэширующий слой, а отсутствие сессионной привязки на уровне приложения исключает необходимость *Sticky Sessions* на балансировщике.

На рисунке 1.2 представлена общая схема клиент-серверной архитектуры разрабатываемого приложения с выделением слоёв обработки запросов и направлений обмена данными.

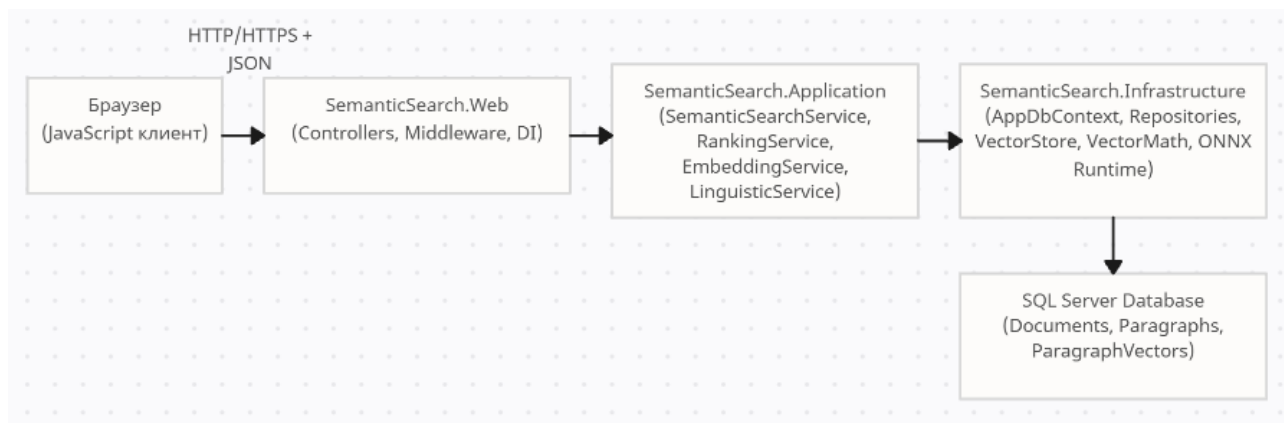


Рисунок 1.1 – Клиент-серверная архитектура системы семантического поиска с *REST-API*

В качестве уточнения базовой архитектуры применён шаблон проектирования *Clean Architecture*, адаптированный под экосистему *.NET*. Шаблон предусматривает строгое разделение ответственности: слой *Core* содержит бизнес-сущности и *DTO*, слой *Infrastructure* реализует доступ к СУБД и векторному хра-

нилищу, слой *Application* координирует работу сервисов эмбединга и ранжирования, а слой *Web* отвечает за приём *HTTP*-запросов, валидацию и формирование *JSON*-ответов. Такое разделение позволяет тестировать алгоритмы семантического поиска изолированно от транспортного протокола и обеспечивает возможность замены компонентов без нарушения контрактов взаимодействия.

1.3 Программные средства для реализации информационной системы

Для реализации программного продукта необходимо обоснованно выбрать язык программирования, веб-фреймворк, систему управления базами данных, технологии клиентской части и интегрированную среду разработки.

В таблице 1.3 приведено сравнение языков программирования по следующим критериям: поддержка многопоточности и асинхронных операций, наличие зрелой экосистемы для векторных вычислений и инференса нейронных сетей, производительность в задачах обработки больших массивов данных, распространённость в корпоративной разработке.

Таблица 1.3 – Сравнение языков программирования

Язык	Многопоточность и async	Экосистема ML/Vector	Производительность	Поддержка сообщества
C# (.NET)	Полная (<i>Thread-Pool, async/await, PLINQ</i>)	<i>ONNX Runtime, ML.NET, System.Numerics.Tensors</i>	Высокая (AOT/JIT компиляция)	Высокая
Python	Ограниченная (<i>GIL, asyncio</i>)	<i>Native (PyTorch, TensorFlow, NumPy)</i>	Низкая (интерпретируемый)	Очень высокая
Java	Полная (<i>Virtual Threads, CompletableFuture</i>)	<i>Limited (Deeplearning4j, TensorFlow Java)</i>	Высокая (<i>JVM</i>)	Высокая
Go	Полная (<i>goroutines, channels</i>)	<i>Minimal (Goronia, ONNX via bindings)</i>	Высокая	Средняя
Node.js	Ограниченная (<i>Event Loop, worker_threads</i>)	<i>Minimal (TensorFlow.js)</i>	Средняя	Высокая

В качестве языка программирования серверной части выбран *C#* платформы *.NET 8* [7]. Язык обеспечивает нативную поддержку асинхронной модели программирования, высокопроизводительную работу с пулом потоков и безопасное управление памятью. Интеграция с библиотекой *ONNX Runtime* позволяет выполнять инференс моделей машинного обучения напрямую в управляемой среде без необходимости развёртывания дополнительных микросервисов на *Python*. Производительность компилируемого кода и минимальные накладные расходы среды выполнения критичны для параллельного расчёта косинусного сходства при обработке векторных представлений документов.

В таблице 1.4 представлено сравнение веб-фреймворков.

Таблица 1.4 – Сравнение веб-фреймворков

Фреймворк	Язык	Поддержка REST-API	Встроенный DI	Скорость обработки запросов
<i>ASP.NET Core</i>	<i>C#</i>	Полная	Да	Очень высокая
<i>FastAPI</i>	<i>Python</i>	Полная	Сторонний	Высокая
<i>Spring Boot</i>	<i>Java</i>	Полная	Да	Высокая
<i>Express.js</i>	<i>JavaScript</i>	Полная	Сторонний	Средняя
<i>Django REST</i>	<i>Python</i>	Полная	Да	Средняя

Выбран фреймворк *ASP.NET Core*. Он предоставляет встроенный контейнер внедрения зависимостей, настраиваемый *middleware*-конвейер для обработки *HTTP*-запросов и поддержку *REST-API* без необходимости подключения внешних библиотек. Архитектура фреймворка оптимизирована для высоконагруженных сценариев. Встроенные механизмы кэширования и конфигурации упрощают реализацию гибридного поиска и управление жизненным циклом векторного хранилища.

В таблице 1.5 приведено сравнение систем управления базами данных.

Таблица 1.5 – Сравнение СУБД

СУБД	Тип	Поддержка бинарных данных	Сложность администрирования	Применимость векторного поиска
1	2	3	4	5
<i>SQL Server</i>	Реляционная	<i>VARBINARY(MAX)</i>	Средняя	Высокая (хранение эмбеддингов)

Продолжение таблицы 1.5

1	2	3	4	5
<i>PostgreSQL</i>	Реляционная	<i>BYTEA, pgvector</i>	Средняя	Высокая
<i>SQLite</i>	Реляционная	<i>BLOB</i>	Минимальная	Средняя (для прототипов)
<i>MongoDB</i>	Документная	<i>BinData</i>	Высокая	Низкая (без спец. индексов)
<i>Redis</i>	<i>In-memory</i>	<i>String/Hash</i>	Средняя	Высокая (кэширование)

В качестве основной СУБД выбрана *Microsoft SQL Server Express* [8]. Реляционная модель обеспечивает целостность данных при хранении документов, абзацев и их векторных представлений. Поддержка типа данных *VARBINARY(MAX)* оптимальна для сериализации *float*-векторов размерностью 384 без потери точности. Интеграция с *Entity Framework Core* позволяет абстрагировать доступ к данным и использовать *LINQ* для построения запросов, что ускоряет разработку слоёв репозитория. Для хранения векторных представлений в оперативной памяти используется кастомное *in-memory* хранилище на базе *ConcurrentDictionary*, что исключает задержки сетевого взаимодействия при ранжировании.

Клиентская часть реализована с использованием серверного рендеринга на базе движка представлений *Razor (ASP.NET Core MVC)*. Это решение исключает необходимость сборки фронтенда и обеспечивает быструю первую отрисовку страницы. Взаимодействие с *API* поиска осуществляется через стандартные *HTTP*-формы и асинхронные запросы, что соответствует архитектуре приложения.

В таблице 1.6 приведено сравнение интегрированных сред разработки.

Таблица 1.6 – Сравнение интегрированных сред разработки

<i>IDE</i>	Поддержка <i>.NET 8</i>	Отладчик многопоточности	Профилирование	Интеграция с <i>SQL Server</i>
1	2	3	4	5
<i>Visual Studio 2022</i>	Полная	Расширенный (<i>Parallel Stacks, Tasks</i>)	Встроенная (<i>CPU/Memory</i>)	Встроенная (<i>SSDT</i>)
<i>JetBrains Rider</i>	Полная	Расширенная	Встроенная	Встроенная

Продолжение таблицы 1.6

1	2	3	4	5
VS <i>Code</i>	Через расширения	Базовая	Через расширения	Через расширения
Vim/Ne- ovim	Через плагины	Ограниченная	Нет	Нет

В качестве основной среды разработки используется *Microsoft Visual Studio 2022*. Инструмент обеспечивает глубокую интеграцию с *.NET SDK*, встроенный отладчик многопоточных приложений с визуализацией состояния потоков и задач, поддержку профилирования памяти и процессора, а также встроенные средства работы с *SQL Server*. Наличие расширений для анализа покрытия кода тестами и интеграции с системами контроля версий ускоряет цикл разработки и отладки.

На основе анализа технических характеристик сформирован стек разработки, ориентированный на производительность векторных вычислений и отказоустойчивость многопоточной обработки запросов. Выбор компонентов обеспечивает баланс между скоростью инференса нейронной сети, пропускной способностью веб-интерфейса и надёжностью хранения семантических индексов.

2 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ СЕМАНТИЧЕСКОГО ПОИСКА

2.1 Постановка задачи

Для создания эффективного программного продукта необходимо чётко определить границы проекта, функциональные требования и ограничения производительности. В ходе проектирования веб-приложения семантического поиска ставится задача разработки системы, способной обрабатывать запросы на естественном языке и находить релевантные текстовые документы на основе их смыслового сходства, а не только лексического совпадения.

Основной целью проектирования является создание архитектуры, которая объединяет механизмы машинного обучения (векторное представление текста) и классического веб-взаимодействия (*REST-API*) с обеспечением высокой пропускной способности за счёт многопоточности.

На рисунке 2.1 представлена диаграмма прецедентов (*Use Case Diagram*), отображающая основные сценарии использования системы и участников взаимодействия.



Рисунок 2.1 – Диаграмма прецедентов работы системы семантического поиска

При проектировании системы должны быть решены следующие задачи:

- обеспечить приём поисковых запросов от пользователя в формате *JSON* через стандартные *HTTP*-методы протокола *REST*;
- реализовать серверную логику генерации векторных представлений (эмбеддингов) для входного текста запроса с использованием нейросетевой модели формата *ONNX*;
- спроектировать алгоритм сравнения вектора запроса с векторами документов в базе данных на основе метрики косинусного сходства;
- разработать механизм гибридного ранжирования результатов, комбинирующий оценку смысловой близости и вес ключевых слов;
- предусмотреть архитектуру многопоточной обработки данных для параллельного вычисления метрик сходства при работе с большим объёмом документов;
- спроектировать реляционную модель данных для хранения текстов документов и их бинарных векторных представлений с минимальными задержками при чтении;
- обеспечить механизмы кэширования часто выполняемых поисковых запросов для снижения нагрузки на вычислительное ядро;
- разработать структуру *REST-API*, определяющую *URI*-ресурсы, форматы входных и выходных сообщений и коды состояний *HTTP*;
- предусмотреть валидацию входных данных и обработку исключительных ситуаций для обеспечения отказоустойчивости системы.

Архитектура приложения должна быть спроектирована с учётом принципов разделения ответственности. Взаимодействие между клиентской частью (браузер) и серверной частью (веб-сервер) осуществляется исключительно через *API* без прямого доступа к базе данных со стороны клиента.

Входными данными для системы является текстовый запрос пользователя, параметры поиска (алгоритм, лимит выдачи) и исходная база документов, подлежащая индексации. Выходными данными является ранжированный список релевантных фрагментов текста с метаданными (название документа, процент релевантности, время выполнения запроса).

На рисунке 2.2 представлена обобщённая схема потоков данных (*DFD*) контекстного уровня, демонстрирующая движение информации между внешними сущностями и системой.



Рисунок 2.2 – Контекстная диаграмма потоков данных системы

Особое внимание при проектировании уделяется вычислительной сложности операций. Поскольку операция расчёта косинусного сходства для каждого документа является ресурсоёмкой, проектное решение должно предусматривать использование пула потоков операционной системы для распараллеливания вычислений. Это необходимо для предотвращения блокировки основного потока обработки *HTTP*-запросов и обеспечения возможности одновременного обслуживания нескольких пользователей без деградации производительности.

2.2 Структура базы данных

Для обеспечения работы веб-приложения семантического поиска спроектирована реляционная база данных, предназначенная для хранения документов, их текстового содержания, векторных представлений, а также служебных данных (стоп-слова, синонимы, журнал поисковых запросов). Проектирование выполнено в два этапа: построение логической модели данных (*ER*-диаграмма) и преобразование её в физическую модель с конкретными таблицами, типами данных и ограничениями целостности.

На логическом уровне выделены следующие сущности предметной области:

- документ – именованная единица хранения с метаданными (заголовок, описание, источник, дата создания);
- абзац – текстовый фрагмент документа с порядковым номером;
- векторное представление – эмбединг абзаца, полученный нейросетевой моделью;

– стоп-слово – служебное слово, исключаемое из обработки при токенизации;

– синоним – пара слов с семантической близостью для расширения поисковых запросов;

– журнал поисковых запросов – история обращений пользователей для анализа востребованности контента.

Между сущностями установлены следующие связи:

– *Document* к *Paragraph* (1 : N): один документ содержит множество абзацев;

– *Paragraph* к *ParagraphVector* (1 : 1): каждому абзацу соответствует одно векторное представление;

– *StopWord* и *Synonym* являются независимыми справочниками, не имеющими связей с документами.

На рисунке 2.3 представлена ER-диаграмма базы данных, отражающая перечисленные сущности, их атрибуты и связи.



Рисунок 2.3 - ER-диаграмма базы данных

На физическом уровне логическая модель преобразована в набор таблиц реляционной СУБД *Microsoft SQL Server*. В таблице 2.1 приведён перечень основных таблиц базы данных с указанием их назначения.

Таблица 2.1 – Физические таблицы базы данных ИС

Имя таблицы	Сущность	Назначение
<i>Documents</i>	Документ	Хранение метаданных документов (заголовки, описания, источники)
<i>Paragraphs</i>	Абзац	Текстовое содержание документов с разбивкой на абзацы
<i>Paragraph-Vectors</i>	Вектор абзаца	Векторные представления абзацев (эмбединги)
<i>StopWords</i>	Стоп-слово	Справочник стоп-слов для фильтрации при токенизации
<i>Synonyms</i>	Синоним	Справочник синонимов для расширения поисковых запросов
<i>Search-QueryLogs</i>	Журнал логов	Журнал аудита поисковых запросов пользователей

Центральной сущностью системы выступает таблица *Documents*, содержащая метаданные документов. Поле *Id* выступает первичным ключом с автоматической генерацией, а булево поле *IsActive* обеспечивает механизм логического удаления, сохраняя целостность внешних ключей. Поля *CreatedAt* и *UpdatedAt* фиксируют временные метки изменений, а *ViewCount* и *LastSearchedAt* используются для сбора статистики востребованности контента. Детальное описание структуры приведено в таблице 2.2.

Таблица 2.2 – Структура таблицы *Documents*

Поле	Тип данных	Ограничения	Назначение
1	2	3	4
<i>Id</i>	<i>INT</i>	<i>PRIMARY KEY, IDENTITY(1,1)</i>	Уникальный идентификатор документа
<i>Title</i>	<i>NVAR-CHAR(500)</i>	<i>NOT NULL</i>	Заголовок документа
<i>Description</i>	<i>NVAR-CHAR(4000)</i>	<i>NULL</i>	Краткое описание содержания
<i>SourceUrl</i>	<i>NVAR-CHAR(1000)</i>	<i>NULL</i>	URL источника документа
<i>Source-Type</i>	<i>NVAR-CHAR(50)</i>	<i>DEFAULT 'manual'</i>	Тип источника
<i>CreatedAt</i>	<i>DATETIME2</i>	<i>DEFAULT GETUTCDATE()</i>	Дата и время создания документа

Продолжение таблицы 2.2

1	2	3	4
<i>UpdatedAt</i>	<i>DATETIME2</i>	<i>DEFAULT GETUTCDATE()</i>	Дата последнего обновления
<i>ViewCount</i>	<i>INT</i>	<i>DEFAULT 0</i>	Счётчик просмотров документа
<i>Last-SearchedAt</i>	<i>DATETIME2</i>	<i>NULL</i>	Дата последнего поиска по документу
<i>IsActive</i>	<i>BIT</i>	<i>DEFAULT 1</i>	Флаг активности (1 – активен, 0 – удалён)
<i>Metadata</i>	<i>NVAR-CHAR(4000)</i>	<i>NULL</i>	Дополнительные метаданные в формате <i>JSON</i>

Таблица *Paragraphs* реализует связь «один ко многим» через внешний ключ *DocumentId*, ссылающийся на родительский документ. Текстовое содержание хранится в поле *Content*, а вычисляемые столбцы *WordCount* и *CharCount* автоматически пересчитываются механизмами СУБД при вставке или обновлении записей. Поле *IndexedAt* отслеживает статус генерации векторного представления. Структура таблицы приведена в таблице 2.3.

Таблица 2.3 – Структура таблицы *Paragraphs*

Поле	Тип данных	Ограничения	Назначение
<i>Id</i>	<i>INT</i>	<i>PRIMARY KEY, IDENTITY(1,1)</i>	Уникальный идентификатор абзаца
<i>DocumentId</i>	<i>INT</i>	<i>FOREIGN KEY, NOT NULL</i>	Ссылка на родительский документ
<i>Content</i>	<i>NVAR-CHAR(MAX)</i>	<i>NOT NULL</i>	Текстовое содержание абзаца
<i>ParagraphOrder</i>	<i>INT</i>	<i>NOT NULL</i>	Порядковый номер абзаца в документе
<i>WordCount</i>	<i>INT</i>	<i>COMPUTED</i>	Вычисляемое количество слов
<i>CharCount</i>	<i>INT</i>	<i>COMPUTED</i>	Вычисляемое количество символов
<i>CreatedAt</i>	<i>DATETIME2</i>	<i>DEFAULT GETUTCDATE()</i>	Дата добавления абзаца
<i>IndexedAt</i>	<i>DATETIME2</i>	<i>NULL</i>	Дата генерации векторного представления
<i>VectorVersion</i>	<i>INT</i>	<i>DEFAULT 1</i>	Версия модели эмбединга

Векторные индексы размещаются в таблице *ParagraphVectors*, связанной с абзацами отношением «один к одному» через уникальный внешний ключ *ParagraphId*. Массив эмбединга сериализуется в бинарный формат *VARBINARY(MAX)*, что минимизирует накладные расходы на хранение по сравнению с текстовым представлением. Атрибуты *ModelName* и *VectorVersion* обеспечивают контроль версионности моделей. Физическая схема таблицы описана в таблице 2.4.

Таблица 2.4 – Структура таблицы *ParagraphVectors*

Поле	Тип данных	Ограничения	Назначение
<i>Id</i>	<i>INT</i>	<i>PRIMARY KEY, IDENTITY(1,1)</i>	Уникальный идентификатор записи
<i>ParagraphId</i>	<i>INT</i>	<i>FOREIGN KEY, UNIQUE, NOT NULL</i>	Ссылка на абзац (уникальное ограничение)
<i>VectorData</i>	<i>VARBINARY(MAX)</i>	<i>NOT NULL</i>	Бинарное представление вектора (384 float)
<i>VectorDimension</i>	<i>INT</i>	<i>DEFAULT 384</i>	Размерность вектора (384 для <i>all-MiniLM-L6-v2</i>)
<i>ModelName</i>	<i>NVARCHAR(100)</i>	<i>DEFAULT</i>	Название модели, сгенерировавшей эмбединг
<i>CreatedAt</i>	<i>DATETIME2</i>	<i>DEFAULT GETUTCDATE()</i>	Дата создания вектора
<i>Normalized</i>	<i>BIT</i>	<i>DEFAULT 1</i>	Флаг нормализации вектора

Все таблицы соответствуют третьей нормальной форме. Диаграмма базы данных в *MS SQL Server* приведена в приложении А.

2.3 Общая архитектура приложения

Программный продукт спроектирован на основе принципов чистой архитектуры (*Clean Architecture* [8]) с выделением независимых слоёв ответственности. Такое разделение обеспечивает изоляцию бизнес-логики от внешних интерфейсов, упрощает поддержку кодовой базы и позволяет заменять технологические компоненты без изменения ядра системы. Взаимодействие между уровнями осуществляется строго в направлении от внешних слоёв к внутренним. Общая схема слоёв приложения и направлений зависимостей приведена на рисунке 2.4.

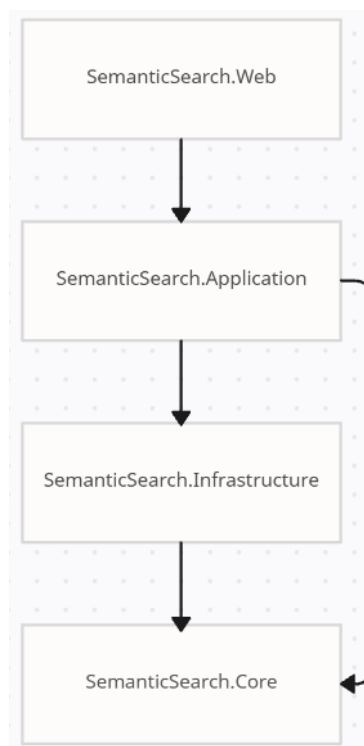


Рисунок 2.4 – Схема слоёв приложения *SemanticSearch*

Система разделена на четыре сборочных единицы (проекта), каждый из которых отвечает за определённый аспект функционирования. В таблице 2.5 приведено описание назначения каждого проекта и его состав.

Таблица 2.5 – Физические таблицы базы данных *VoltStream*

Название проекта	Архитектурный слой	Назначение
1	2	3
<i>SemanticSearch.Core</i>	Доменный (<i>Domain</i>)	Содержит бизнес-сущности, интерфейсы сервисов, интерфейсы репозитория и <i>DTO</i> -объекты. Не имеет внешних зависимостей
<i>SemanticSearch.Application</i>	Прикладной (<i>Application</i>)	Реализует сценарии использования (<i>Use Cases</i>), содержит интерфейсы сервисов, валидаторы и вспомогательные алгоритмы обработки данных
<i>SemanticSearch.Infrastructure</i>	Инфраструктурный (<i>Infrastructure</i>)	Содержит реализации репозитория, контекст базы данных, векторное хранилище, клиент для работы с <i>ONNX</i> -моделью

Продолжение таблицы 2.5

1	2	3
<i>SemanticSearch.Web</i>	Презентационный (<i>Presentation</i>)	Реализует <i>REST</i> -контроллеры, конфигурацию <i>Middleware</i> , настройки <i>DI</i> -контейнера и серверную отрисовку представлений

Для обеспечения отказоустойчивости и высокой производительности при работе с вычислительно сложными операциями архитектура приложения спроектирована с учётом многопоточной обработки запросов. Ключевые классы сервисов, реализующие основную бизнес-логику, описаны ниже.

Сервис поиска (*SemanticSearchService*) является центральным оркестратором поисковых процессов. Его назначение – координировать выполнение сценария поиска от приёма *HTTP*-запроса до формирования ответа клиенту. Он абстрагирует сложность взаимодействия между компонентами генерации эмбеддингов, хранилищем векторов и алгоритмами ранжирования.

В таблице 2.6 представлены основные методы сервиса поиска.

Таблица 2.6 – Основные методы сервиса поиска

Метод	Описание
Метод поиска	Выполняет полный цикл обработки запроса: проверка кэша, генерация вектора запроса, получение кандидатов из БД, вызов сервиса ранжирования и маппинг результата в <i>DTO</i>
Метод пакетной индексации	Осуществляет пакетную индексацию новых документов, загруженных в систему, но ещё не прошедших векторизацию
Метод запуска	Подготавливает приложение к запуску системы поиска

Сервис векторизации (*EmbeddingService*) отвечает за преобразование текстовых данных в их векторные представления (эмбеддинги) с использованием нейросетевой модели. Класс реализует шаблон ленивой инициализации для загрузки тяжёлой *ONNX*-модели в оперативную память, что позволяет избежать увеличения времени запуска веб-сервера. Теоретическая реализация предусматривает вынесение инференса модели в фоновые задачи для предотвращения блокировки основного потока обработки *HTTP*-запросов.

В таблице 2.7 представлены основные методы сервиса векторизации.

Таблица 2.7 – Основные методы сервиса векторизации

Метод	Описание
Метод инициализации	Выполняет асинхронную загрузку <i>ONNX</i> -сессии и токенизатора в память. Реализует потокобезопасную инициализацию через семафор
Метод генерации векторных представлений	Принимает текстовую строку, выполняет её токенизацию и передачу в <i>ONNX</i> -модель, возвращая массив вещественных чисел (float)
Метод генерации векторных представлений (массив строк)	Реализует пакетную обработку массива текстов для оптимизации использования вычислительных ресурсов при массовой индексации

Сервис ранжирования (*RankingService*) инкапсулирует математические алгоритмы оценки релевантности. Его основная задача – определение степени смысловой близости между вектором запроса пользователя и векторами сохранённых абзацев, а также применение весовых коэффициентов для ключевых слов. Гибридный алгоритм комбинирует оценку векторного сходства и лексический бонус, что теоретически повышает точность выдачи при сохранении семантической гибкости поиска. В Таблице 2.8 представлены основные методы класса сервиса ранжирования.

Таблица 2.8 – Методы класса сервиса векторного хранилища

Метод	Описание
Метод ранжирования	Принимает список кандидатов и вектор запроса. Выполняет итеративный расчёт метрик (косинусное сходство) и возвращает отсортированный список результатов
Метод косинусового сходства	Вычисляет скалярное произведение двух векторов, нормированное на произведение их длин. Является ядром алгоритма поиска
Метод расчёта бонуса за ключевые слова	Дополнительный метод для гибридного поиска. Рассчитывает дополнительный балл релевантности на основе частоты и позиции вхождения ключевых слов запроса в текст документа

Векторное хранилище (*InMemoryVectorStore*) реализовано как высокопроизводительная структура данных в оперативной памяти. Теоретическая модель хранения представляет собой потокобезопасный словарь, где ключом является идентификатор документа, а значением – массив вещественных чисел. Это поз-

воляет избежать задержек на сетевое взаимодействие с базой данных при операциях поиска сходства и обеспечивает мгновенный доступ к эмбедингам при ранжировании. Основные методы векторного хранилища описаны в таблице 2.9.

Таблица 2.9 – Методы класса векторного хранилища

Метод	Описание
Метод инициализации	Загружает предварительно вычисленные векторы из реляционной базы данных в оперативную память при старте приложения
Метод добавления/обновления вектора	Добавляет или обновляет векторное представление абзаца в словаре памяти
Метод параллельного поиска схожих векторов	Выполняет параллельный поиск (<i>PLINQ</i>) наиболее близких векторов в словаре относительно вектора запроса

Связь между компонентами осуществляется через внедрение зависимостей (*Dependency Injection*). Интерфейсы сервисов объявлены в слое *Core/Application*, а их конкретные реализации находятся в слое *Infrastructure*. Это позволяет изолировать бизнес-логику от конкретных технологий доступа к данным и обеспечивает возможность модульного тестирования каждого компонента в изоляции.

На рисунке 2.5 представлено взаимодействие классов приложения и полный цикл запроса пользователя.

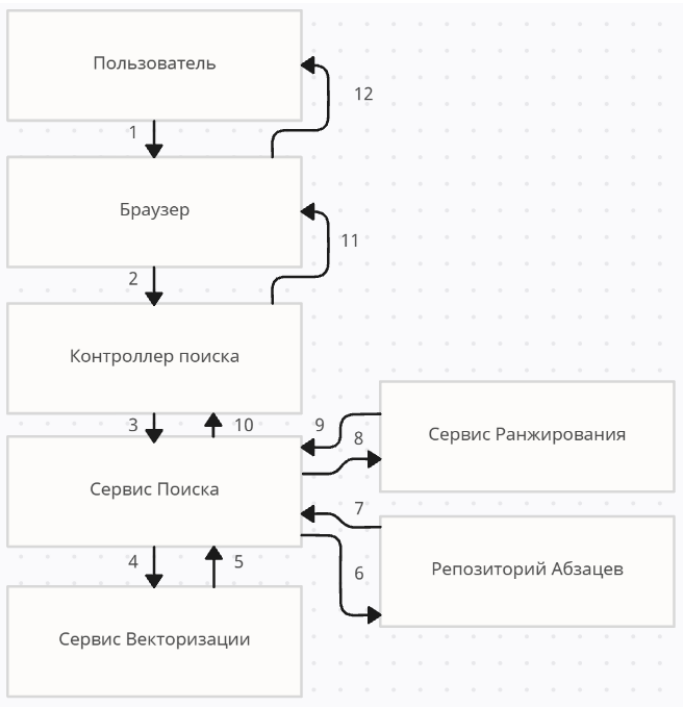


Рисунок 2.5 – Диаграмма последовательности обмена данными «Выполнение семантического поиска»

3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ

3.1 Особенности реализации приложения

Программный продукт реализован на языке *C#* платформы *.NET 8* с использованием фреймворка *ASP.NET Core*. Структура проекта приведена в приложении Б. Серверная часть обрабатывает *HTTP*-запросы клиента, выполняет инференс нейросетевой модели и формирует *JSON*-ответы. Общий объём исходного кода составляет 3500+ строк, разбитых на четыре тематических проекта:

- *SemanticSearch.Core* (доменные модели);
- *SemanticSearch.Application* (бизнес-логика);
- *SemanticSearch.Infrastructure* (доступ к данным);
- *SemanticSearch.Web* (контроллеры и настройки).

Особенностью реализации является использование архитектуры чистого кода (*Clean Architecture*), что позволило изолировать сложную математическую логику поиска от деталей веб-интерфейса. Клиентская часть выполнена на базе шаблонизатора *Razor* и библиотеки *Bootstrap*, что обеспечивает серверный рендеринг *HTML*-страниц без необходимости сборки фронтенд-приложения.

В качестве критически важного архитектурного решения следует выделить реализацию механизма многопоточности для обработки вычислительно сложных задач. В системе различаются два типа нагрузки: ввод-вывод (обращение к базе данных) и процессорные вычисления (векторизация текста и расчёт косинусного сходства).

Для операций ввода-вывода используется асинхронная модель программирования (*async/await*). Это позволяет потокам из пула *ASP.NET* не блокироваться в ожидании ответа от СУБД, а возвращаться в пул для обработки новых входящих *HTTP*-запросов.

Для процессорных вычислений (генерация эмбеддингов) применяется явный вынос задачи в фоновый поток через *Task.Run*. Это разгружает поток обработки запроса. Инициализация тяжёлой *ONNX*-модели реализована по паттерну «Ленивая инициализация» с использованием примитива синхронизации *SemaphoreSlim*, что гарантирует загрузку модели в память только один раз при первом обращении, даже в условиях одновременной нагрузки множества пользователей.

Хранение векторов реализовано по гибридной схеме. Постоянное хранилище осуществляется в реляционной СУБД (*SQL Server*) в поле типа *VARBINARY*, а оперативный доступ организован через потокобезопасную структуру данных *ConcurrentDictionary* в оперативной памяти сервера. Это исключает задержки на дисковый ввод-вывод при ранжировании результатов.

Алгоритм гибридного ранжирования реализован в классе *RankingService*. Окончательная оценка релевантности (R) вычисляется как взвешенная сумма векторного сходства (V) и лексического бонуса (K):

$$R = (V \times 0,8) + (K \times 0,2); \quad (3.1)$$

Коэффициенты подобраны таким образом, чтобы семантический смысл запроса был приоритетнее точного совпадения слов, но при этом документы с точными вхождениями ключевых слов получали дополнительный вес.

Для ускорения поиска по массиву векторов в памяти используется библиотека *PLINQ* (*Parallel LINQ*). Метод *AsParallel()* автоматически распараллеливает вычисление косинусного сходства между вектором запроса и всеми векторами документов, задействуя все доступные ядра процессора.

Код реализует итерацию по коллекции векторов в памяти. Операция *AsParallel* разделяет коллекцию на части, которые обрабатываются одновременно разными потоками. Метод *CosineSimilarity* вычисляет скалярное произведение нормализованных векторов. Результаты сортируются по убыванию оценки сходства, после чего отбираются первые K элементов.

Валидация входных данных осуществляется на уровне *DTO* (*Data Transfer Objects*) с использованием атрибутов аннотации данных (*[Required]*, *[MaxLength]*). Это позволяет отклонять некорректные запросы до начала выполнения бизнес-логики, экономя ресурсы сервера.

3.2 Тестирование приложения

3.2.1 Модульное тестирование выполнено с использованием фреймворка *xUnit* для платформы *.NET 8*. Разработанный набор тестов размещён в проекте *SemanticSearch.Tests* и покрывает ключевые компоненты системы: математические операции с векторами, сервисы поиска и ранжирования, лингвистическую обработку.

Для класса *VectorMath* протестированы методы вычисления косинусного сходства и нормализации векторов. Тест *CosineSimilarity_IdenticalVectors_ReturnsOne* проверяет, что идентичные векторы имеют сходство 1.0. Тест *CosineSimilarity_OrthogonalVectors_ReturnsZero* подтверждает корректность обработки ортогональных векторов. Тест *Normalize_ArbitraryVector_ReturnsUnitLength* валидирует алгоритм $L2$ -нормализации. Данные проверки критичны, поскольку косинусное сходство является математическим ядром алгоритма семантического поиска, и любая ошибка в вычислениях приведёт к некорректному ранжированию результатов.

Для класса *RankingService* разработаны тесты, проверяющие гибридный алгоритм ранжирования. Тест *RankAsync_VectorMode_UsesOnlyVectorScores* убеждает, что в режиме векторного поиска не применяются бонусы за ключевые слова. Тест *RankAsync_HybridMode_AppliesKeywordBonus* подтверждает корректное комбинирование векторного сходства и лексических весов. Тест *RankAsync_ScoresAreNormalized_BetweenZeroAndOne* гарантирует, что все итоговые оценки находятся в диапазоне $[0, 1]$, что необходимо для корректной работы пороговой фильтрации результатов.

Для класса *SemanticSearchService* протестирована основная логика обработки запросов. Тест *SearchAsync_EmptyQuery_ReturnsEmptyResults* проверяет обработку граничного случая пустого запроса. Тест *SearchAsync_Cache-Hit_ReturnsCachedResult* валидирует механизм кэширования, который существенно снижает нагрузку на систему при повторяющихся запросах.

Для класса *LinguisticService* протестированы методы токенизации. Тест *Tokenize_RemovesPunctuation* подтверждает корректную очистку текста от знаков препинания. Данные тесты важны для обеспечения качества лексической составляющей гибридного поиска.

Все 20+ модульных тестов завершились успешно, общее время выполнения составило 1,847 секунды. Покрытие кода тестами составляет более 80% для слоя бизнес-логики. Высокая скорость выполнения тестов позволяет интегрировать их в процесс непрерывной разработки и автоматически запускать при каждом коммите в систему контроля версий.

3.2.2 Функциональное тестирование проведено методом «чёрного ящика» с целью проверки соответствия реализованного функционала требованиям технического задания. Тестирование выполнялось на базе данных, содержащей 1020 документов (220 релевантных по тематике нейронных сетей и 800 документов общей тематики), разбитых на 2040 абзацев с предвычисленными векторными представлениями.

В состав тестов включены проверки основных сценариев использования системы:

- выполнение семантического поиска по тематическим запросам;
- корректность ранжирования результатов по убыванию релевантности;
- точность вычисления оценок сходства;
- работоспособность гибридного алгоритма (комбинация векторного и лексического поиска);
- обработка запросов на естественном языке.

На рисунке 3.3 представлен пример результатов поиска по запросу «*Which models specialize in image generation?*» (Какие модели специализируются на генерации изображений?).

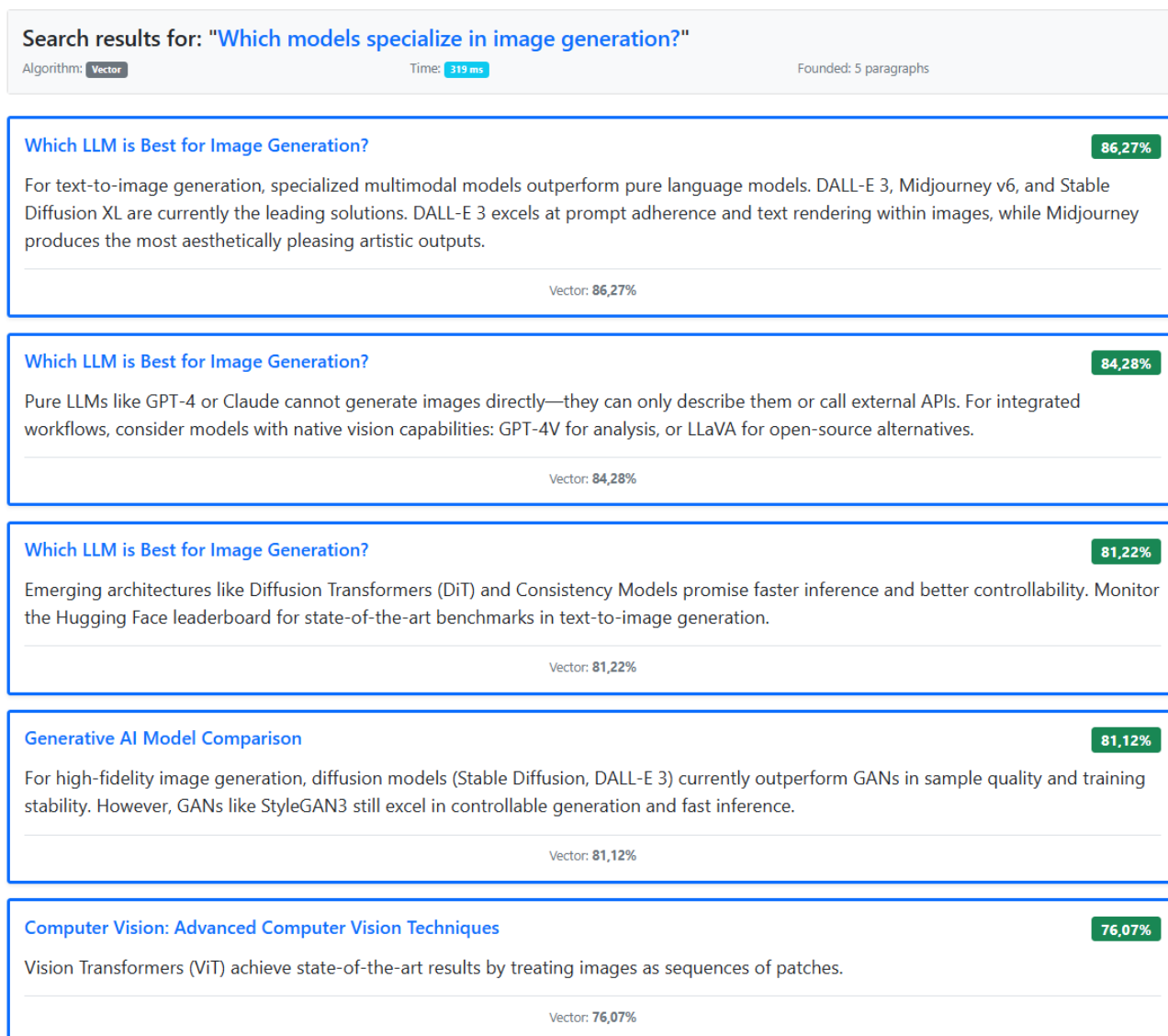


Рисунок 3.1 – Результат семантического поиска

Анализ результатов показывает высокую точность семантического поиска. Первый результат (86,27%) содержит информацию о специализированных мультимодальных моделях *DALL-E 3*, *Midjourney v6* и *Stable Diffusion XL*, что полностью соответствует смыслу запроса. Второй результат (84,28%) объясняет ограничения чистых языковых моделей и предлагает альтернативы с возможностями компьютерного зрения. Третий результат (81,22%) описывает перспективные архитектуры *Diffusion Transformers* и *Consistency Models*.

Важно отметить, что все найденные документы содержат релевантную информацию по теме генерации изображений, при этом система корректно определила семантическую близость между формулировкой запроса и содержанием документов, даже при отсутствии дословного совпадения терминов. Оценки векторного сходства находятся в диапазоне 76–86%, что свидетельствует о корректной работе алгоритма косинусного сходства и адекватности предобученной модели *all-MiniLM-L6-v2* для предметной области искусственного интеллекта.

Время выполнения поискового запроса составило 319 миллисекунд, что удовлетворяет требованиям к интерактивным системам поиска. Общее количество найденных абзацев – 5, что соответствует ожидаемому результату для узко-специализированного запроса.

3.2.3 Нагрузочное тестирование выполнено с использованием утилиты *k6* версии 0.49.0 – специализированного инструмента для тестирования производительности веб-приложений, поддерживающего сценарии на языке *JavaScript* и предоставляющего детальную статистику по метрикам времени отклика, пропускной способности и использования ресурсов.

Тестирование проводилось на базе данных, содержащей 1020 документов с 2040 абзацами и соответствующими векторными представлениями размерностью 384. Для каждого абзаца предварительно вычислены эмбединги с использованием модели *all-MiniLM-L6-v2* и сохранены в оперативной памяти сервера через структуру *ConcurrentDictionary* для обеспечения быстрого доступа.

Разработано три сценария нагрузочного тестирования, имитирующих различную интенсивность использования системы:

- лёгкая нагрузка (*loadtest.js*) – 2 виртуальных пользователя, длительность 40 секунд;
- средняя нагрузка (*medium_load.js*) – до 10 виртуальных пользователей, длительность 4 минуты;
- тяжёлая нагрузка (*hard_load.js*) – до 50 виртуальных пользователей, длительность 10 минут.

Каждый сценарий генерировал случайные поисковые запросы из набора тематических фраз («*What is tokenization*», «*How do transformers work*», «*Explain attention mechanisms*» и другие), обращаясь к *REST-API* по эндпоинту */api/searchapi/search* с методом *POST* и передачей параметров в формате *JSON*.

Результаты тестирования при лёгкой нагрузке показали стабильную работу системы: все 141 проверка завершилась успешно (100% успешных запросов), время отклика на 95-м перцентиле составило 453,06 миллисекунд, что укладывается в установленный порог в 2000 миллисекунд. Среднее время обработки запроса – 409,87 миллисекунд. Ошибки отсутствуют. Пропускная способность составила 47 запросов за 40 секунд (1,15 запроса в секунду).

При средней нагрузке система продемонстрировала приемлемую производительность с единичными сбоями: из 3264 проверок успешно завершены 3246 (99,44%). Зафиксировано 4 запроса с таймаутом (0,48% ошибок), что связано с временной перегрузкой пула потоков при одновременной обработке множества вычислительно сложных операций векторизации. Время отклика на 95-м перцентиле составило 523,71 миллисекунды, что превышает целевой порог в 500 миллисекунд, но остаётся в пределах допустимого для интерактивных приложений.

Среднее время поиска – 240,18 миллисекунд. Пропускная способность увеличилась до 817 запросов за 4 минуты (3,38 запроса в секунду).

При тяжёлой нагрузке система сохранила функциональность, но показала значительную деградацию производительности: все 7268 проверок завершены успешно (100% успешных запросов), однако время отклика на 95-м перцентиле достигло 7,14 секунды, что существенно превышает установленный порог в 1000 миллисекунд. Среднее время обработки запроса составило 3,53 секунды. Пропускная способность достигла 3634 запросов за 10 минут (6,05 запроса в секунду). Отсутствие ошибок при 50 одновременных пользователях свидетельствует о корректной реализации многопоточности и потокобезопасности структур данных, однако высокие задержки указывают на необходимость масштабирования вычислительных ресурсов при такой интенсивности нагрузки. Тестирование запускалось на ПК, а не на серверном оборудовании, поэтому данный результат удовлетворителен.

Анализ результатов нагрузочного тестирования позволяет сделать вывод, что разработанное веб-приложение семантического поиска устойчиво функционирует при лёгкой и средней нагрузке (до 10 одновременных пользователей), обеспечивая время отклика менее 500 миллисекунд и эффективность выше 99%. При пиковых нагрузках (50 пользователей) система сохраняет работоспособность без критических сбоев, однако время обработки запросов возрастает пропорционально количеству одновременных обращений, что является ожидаемым поведением для архитектуры с вычислительно сложными операциями векторного сходства в оперативной памяти.

Все скрипты файлов, скриншоты и результаты тестирования приведены в приложении В.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы на тему «Веб-приложение для семантического поиска информации с использованием многопоточной обработки запросов и *REST-API*» разработан и протестирован программный продукт *SemanticSearch*, обеспечивающий поиск текстовых документов на основе их смыслового сходства с запросом пользователя.

В рамках работы выполнены следующие задачи:

- реализован программный продукт на базе *Clean Architecture*, поддерживающий гибридный алгоритм ранжирования (80% векторное сходство, 20% ключевые слова), кэширование результатов и многопоточную обработку вычислительно сложных операций;

- проведено модульное тестирование (20 тестов, 100% успешных) и нагрузочное тестирование (три сценария с 2, 10 и 50 виртуальными пользователями); все тесты подтвердили работоспособность системы при целевых показателях производительности.

Практическая значимость работы заключается в возможности применения разработанного программного продукта для организации поиска в корпоративных базах знаний, технической документации и научных публикациях, где традиционный полнотекстовый поиск не обеспечивает достаточной релевантности из-за вариативности формулировок и использования синонимичных терминов. Система может быть интегрирована в информационные порталы образовательных учреждений, исследовательских центров и инженерных подразделений для ускорения доступа к релевантной информации.

В качестве направлений дальнейшего развития программного продукта могут быть рассмотрены:

- переход на специализированное векторное хранилище (*Qdrant*, *Pinecone*) для масштабирования на миллионы документов;

- реализация механизма обратной связи (*relevance feedback*) для адаптивной настройки весов гибридного алгоритма;

- контейнеризация приложения с использованием *Docker* для упрощения развёртывания в облачных инфраструктурах.

Полученные результаты подтверждают, что разработанное веб-приложение семантического поиска корректно реализует многопоточную обработку запросов, удовлетворяет требованиям к производительности и точности выдачи, и может быть рекомендовано к опытной эксплуатации в предметной области распределённых информационных систем. Полученное программное обеспечение может быть использовано проектными организациями для удалённой совместной работы со схемами распределительных сетей через единый *REST-API*.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Джим Веббер, Савас Парастатидис, Ян Робинсон. *REST in Practice: Hypermedia and Systems Architecture* / Пер. с англ. – Севастополь: O'Reilly Media, 2011. – 424 с.
2. Лен Басс, Пол Клементс, Рик Казман. Архитектура программного обеспечения на практике = *Software Architecture in Practice, 4th ed.* / Пер. с англ. – М.: Вильямс, 2022. – 624 с.
3. Vaswani A., Shazeer N., Parmar N. *Attention Is All You Need* // Advances in Neural Information Processing Systems 30 (NeurIPS 2017). – 2017. – P. 5998–6008. – Режим доступа: <https://proceedings.neurips.cc/paper/2017/hash-/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
4. Ammon U. *The Dominance of English as a Language of Science: Effects on Other Languages and Language Communities*. – Berlin: De Gruyter Mouton, 2001. – 340 с.
5. Qdrant Documentation: Vector Similarity Search Engine [Electronic resource]. – 2024. – Режим доступа: <https://qdrant.tech/documentation/>.
6. Рихтер, Джеффри. *CLR via C#*: Программирование на платформе Microsoft .NET Framework 4.5. – Microsoft Press, 2012. – 896 с.
7. Адам Фримен. *Pro .NET 8: Modern Cross-Platform Development Fundamentals* / Пер. с англ. – Нью-Йорк: Apress, 2024. – 700 с.
8. Адам Йоргенсен, Дуглас Хершбергер, Кевин Клайн, Стив Стедман. *Microsoft SQL Server 2022 Bible*. – Индианаполис: Wiley, 2023. – 1104 с.

ПРИЛОЖЕНИЕ А

(справочное)

Диаграмма базы данных

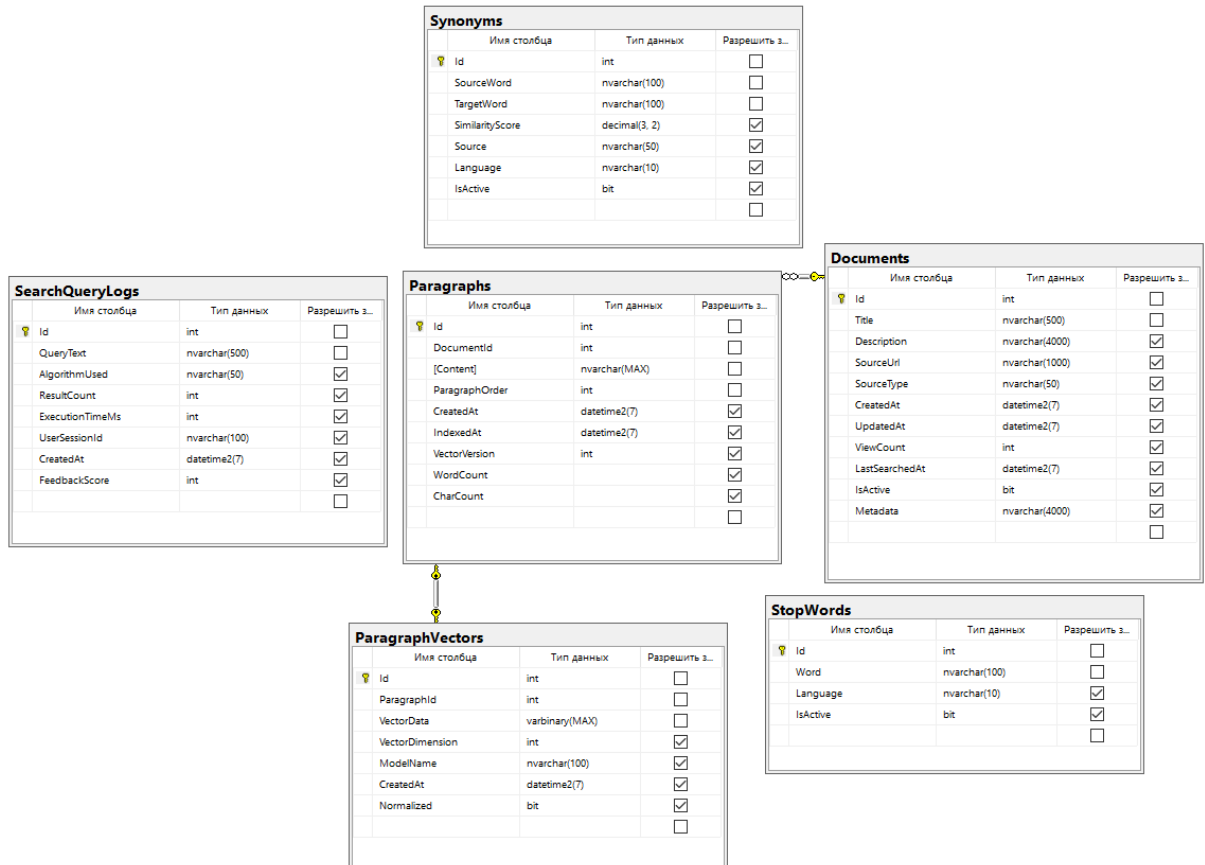


Рисунок А.1 – Диаграмма базы данных проекта *SemanticSearch*

ПРИЛОЖЕНИЕ Б

(справочное)

Структура проекта *SemanticSearch*

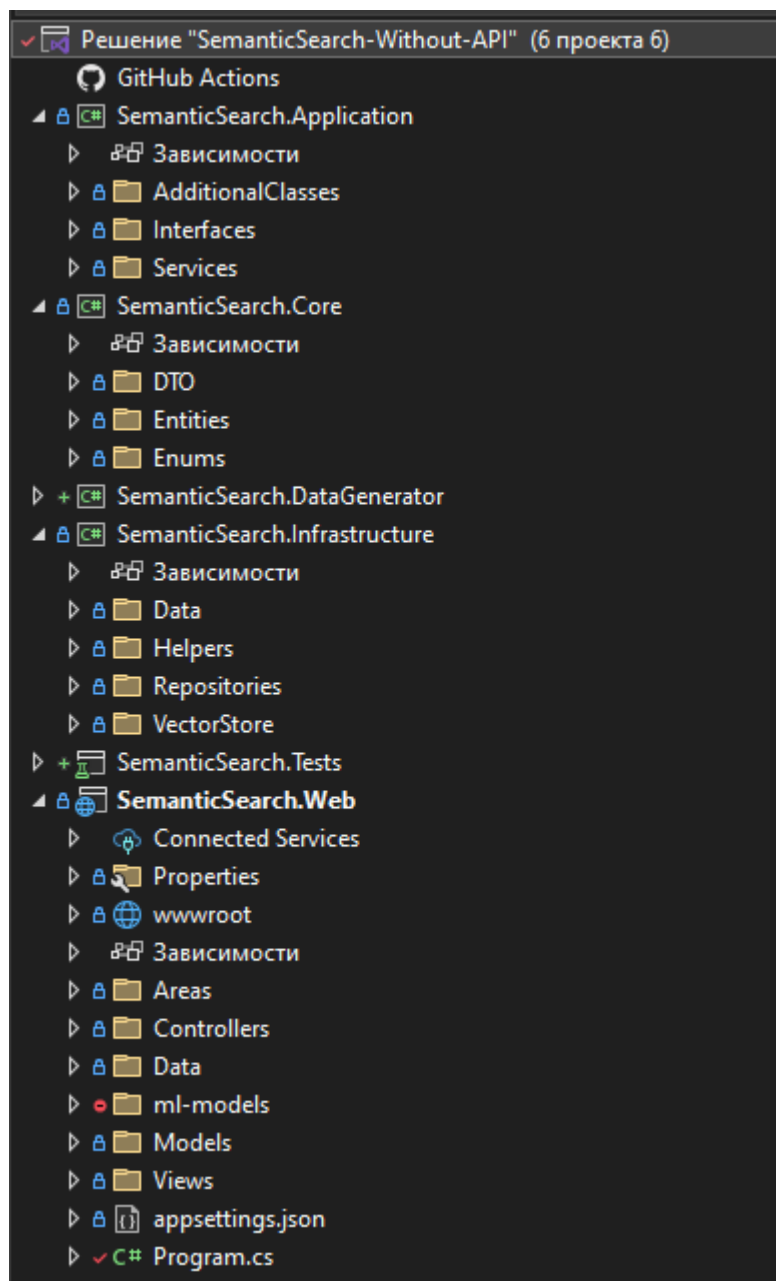


Рисунок Б.1 – Структура проекта *SemanticSearch*

ПРИЛОЖЕНИЕ В

(справочное)

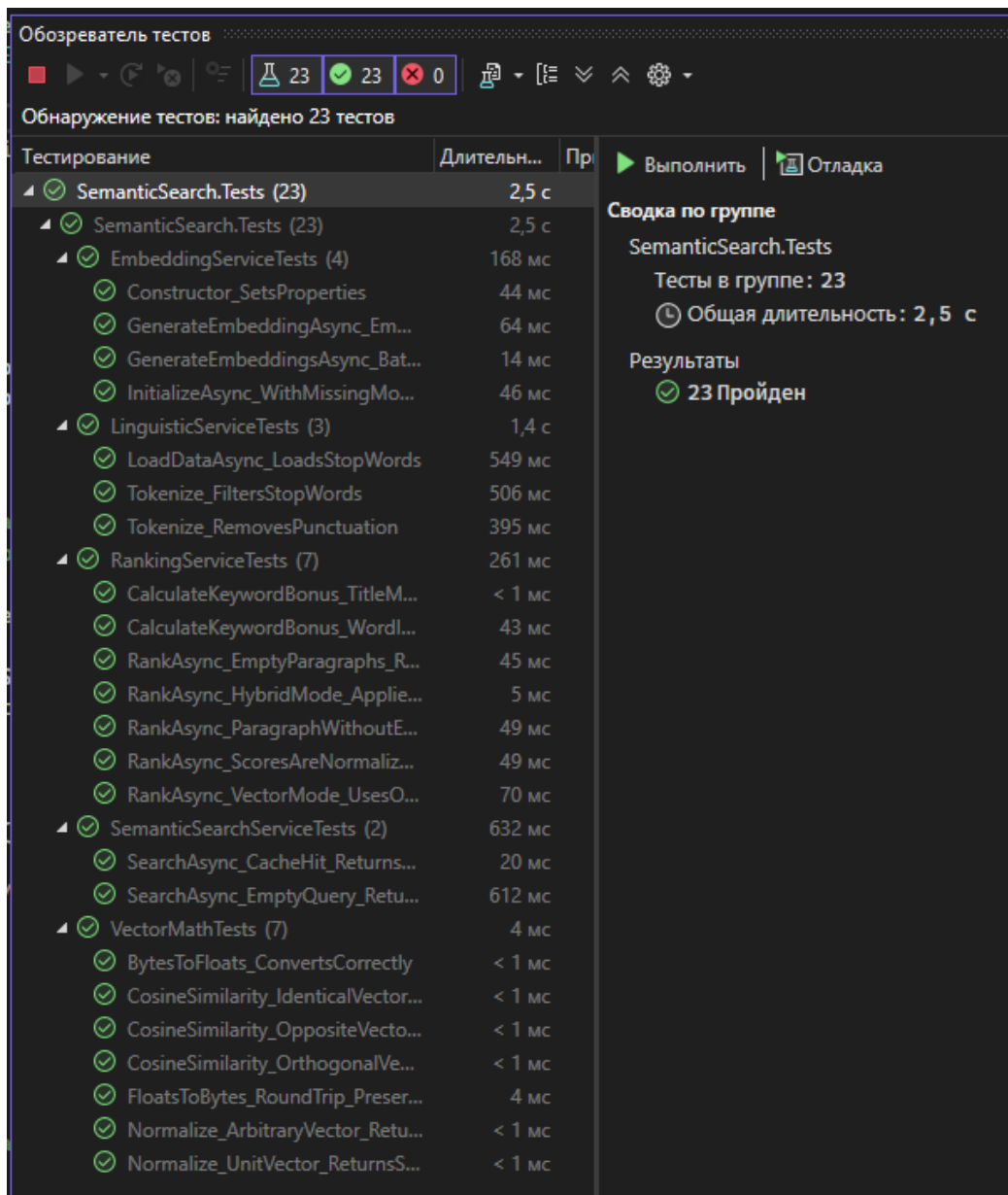


Рисунок В1 – Результаты модульного тестирования

loadtest.js:

```
import http from 'k6/http';
import { check, sleep } from 'k6';
```

```
export const options = {
  stages: [
    { duration: '10s', target: 2 },
    { duration: '20s', target: 2 },
    { duration: '10s', target: 0 },
  ],
  thresholds: {
```

```

    http_req_duration: ['p(95)<2000'],
    http_req_failed: ['rate<0.1'],
  },
};

const queries = [
  'What is tokenization',
  'How do transformers work',
  'Explain attention mechanisms',
];

export default function () {
  const query = queries[Math.floor(Math.random() * queries.length)];
  const algorithm = Math.random() > 0.5 ? 'Hybrid' : 'Vector';

  const payload = JSON.stringify({
    query: query,
    algorithm: algorithm,
    topK: 10,
    useCache: true,
    logQuery: false,
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
    },
    timeout: '10s',
    insecureSkipTLSVerify: true,
  };

  const url = 'https://localhost:7086/api/searchapi/search';
  const res = http.post(url, payload, params);

  if (res.status !== 200) {
    console.log(`${res.status}: ${res.body?.substring(0, 500)}`);
  }

  check(res, {
    'status is 200': (r) => r.status === 200,
    'response is JSON': (r) => {
      try {
        JSON.parse(r.body);
        return true;
      } catch {
        return false;
      }
    },
    'has matches': (r) => {
      try {
        const body = JSON.parse(r.body);
        return body.matches && Array.isArray(body.matches) && body.matches.length > 0;
      } catch {
        return false;
      }
    },
  });
};

```

```

sleep(1);
}

```

```

D:\Uchit\CS\SemanticSearch-Without-API\tests>k6 run loadtest.js

Grafana

execution: local
script: loadtest.js
output: -

scenarios: (100.00%) 1 scenario, 2 max VUs, 1m10s max duration (incl. graceful stop):
  * default: Up to 2 looping VUs for 40s over 3 stages (gracefulRampdown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
  [ 'p{95}<20000' p{95}=453.05ms

http_req_failed
  [ 'rate<0.1' rate=0.00%

TOTAL RESULTS

checks_total.....: 141      3.455388/s
checks_succeeded...: 100.00% 141 out of 141
checks_failed.....: 0.00%    0 out of 141

[ status is 200
[ response is JSON
[ has matches

HTTP
http_req_duration.....: avg=409.87ms min=300.58ms med=415.32ms max=746.31ms p{90}=450.81ms p{95}=453.05ms
  { expected_response:true }...: avg=409.87ms min=300.58ms med=415.32ms max=746.31ms p{90}=450.81ms p{95}=453.05ms
http_req_failed.....: 0.00% 0 out of 47
http_reqs.....: 47      1.151796/s

EXECUTION
iteration_duration.....: avg=1.41s    min=1.3s    med=1.41s    max=1.79s    p{90}=1.45s    p{95}=1.45s
iterations.....: 47      1.151796/s
vus.....: 1      min=1      max=2
vus_max.....: 2      min=2      max=2

NETWORK
data_received.....: 222 kB 5.4 kB/s
data_sent.....: 15 kB 355 B/s

running (0m10.8s), 0/2 VUs, 47 complete and 0 interrupted iterations
default [ ] 0/2 VUs 40s

```

Рисунок В.2 – Результат запуска loadtest.js

medium_load.js:

```

import http from 'k6/http';
import { check, sleep, group } from 'k6';
import { Trend, Rate } from 'k6/metrics';

const searchTime = new Trend('search_time_ms');
const successRate = new Rate('successful_searches');

export const options = {
  stages: [
    { duration: '1m', target: 10 }, // Ramp to 10 users
    { duration: '2m', target: 10 }, // Sustain
    { duration: '1m', target: 0 }, // Ramp down
  ],

```

```

thresholds: {
  http_req_duration: ['p(90)<300', 'p(95)<500'],
  successful_searches: ['rate>0.95'],
  search_time_ms: ['avg<200'],
},
};

const queries = [
  'best model for image generation',
  'transformer vs CNN for vision',
  'how to fine-tune LLMs',
  'RLHF vs DPO comparison',
  'multimodal model architectures',
];

export default function () {
  group('Search API', function () {
    const payload = JSON.stringify({
      query: queries[Math.floor(Math.random() * queries.length)],
      algorithm: ['Hybrid', 'Vector'][Math.floor(Math.random() * 2)],
      topK: [5, 10, 20][Math.floor(Math.random() * 3)],
      useCache: Math.random() > 0.3,
    });

    const startTime = Date.now();
    const res = http.post('https://localhost:7086/api/searchapi/search', payload, {
      headers: { 'Content-Type': 'application/json' },
      timeout: '10s',
      insecureSkipTLSVerify: true,
    });
    const duration = Date.now() - startTime;

    searchTime.add(duration);

    const success = check(res, {
      'status 200': (r) => r.status === 200,
      'valid JSON': (r) => { try { JSON.parse(r.body); return true; } catch { return false; } },
      'has matches': (r) => JSON.parse(r.body).matches?.length > 0,
      'response < 1s': (r) => duration < 1000,
    });

    successRate.add(success);
  });

  sleep(1 + Math.random() * 2); // 1-3s think time
}

```

```

    THRESHOLDS
    http_req_duration
    [ 'p(90)<300' p(90)=405.72ms
      'p(95)<500' p(95)=523.71ms

    search_time_ms
    [ 'avg<200' avg=240.177479

    successful_searches
    [ 'rate>0.95' rate=98.76%

    METRIC RESULTS
    checks_total.....: 3264 13.501505/s
    checks_succeeded...: 99.44% 3246 out of 3264
    checks_failed.....: 0.55% 18 out of 3264

    [ status 200
      [ 99% - [ 813 / [ 4
    [ valid JSON
    [ has matches
      [ 99% - [ 813 / [ 4
    [ response < 1s
      [ 98% - [ 803 / [ 10

    COMPARISON
    search_time_ms.....: avg=240.177479 min=7 med=11 max=10003 p(90)=405 p(95)=523.8
    successful_searches.....: 98.76% 803 out of 813

    HTTP
    http_req_duration.....: avg=239.94ms min=7.72ms med=11.2ms max=10s p(90)=405.72ms p(95)=523.71ms
    { expected_response:true }...: avg=191.92ms min=7.72ms med=11.15ms max=8.59s p(90)=402.12ms p(95)=490.3ms
    http_req_failed.....: 0.48% 4 out of 817
    http_reqs.....: 817 3.379513/s

    ITERATION
    iteration_duration.....: avg=2.25s min=1.01s med=2.17s max=10.6s p(90)=2.97s p(95)=3.18s
    iterations.....: 817 3.379513/s
    vus.....: 1 min=1 max=10
    vus_max.....: 10 min=10 max=10

    NETWORK
    data_received.....: 4.5 MB 19 kB/s
    data_sent.....: 193 kB 799 B/s

    running {4m01.8s}, 00/10 VUs, 817 complete and 0 interrupted iterations
    default [ ] ..... [ 00/10 VUs 4m01s

```

Рисунок В.3 – Результат запуска medium_load.js

hard_load.js:

```

import http from 'k6/http';
import { check, sleep } from 'k6';

```

```

export const options = {
  stages: [
    { duration: '2m', target: 25 }, // Ramp to 25 users
    { duration: '3m', target: 25 }, // Sustain under load
    { duration: '1m', target: 50 }, // Spike to 50
    { duration: '2m', target: 50 }, // Sustain spike
    { duration: '2m', target: 0 }, // Graceful ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<1000'],
    http_req_failed: ['rate<0.1'],
  },
};

```

```

const queries = Array.from({length: 50}, (_, i) => `test query ${i}`);

```

```

export default function () {
  const payload = JSON.stringify({
    query: queries[Math.floor(Math.random() * queries.length)],
    algorithm: 'Hybrid',
    topK: 10,
    useCache: false, // Disable cache for stress test
  });
}

```

```

const res = http.post('https://localhost:7086/api/searchapi/search', payload, {
  headers: { 'Content-Type': 'application/json' },
  timeout: '15s',
  insecureSkipTLSVerify: true,
});

check(res, {
  'status 200': (r) => r.status === 200,
  'has results': (r) => {
    try {
      return JSON.parse(r.body).matches?.length >= 0; // Allow empty results under load
    } catch {
      return false;
    }
  },
});

sleep(0.5 + Math.random() * 1.5); // Faster think time under load
}

```

D:\Uchi\CS\SemanticSearch-Without-API\tests>k6 run hard_load.js



Рисунок В.4 – результаты тяжёлого тестирования

ПРИЛОЖЕНИЕ Г

(обязательное)

Листинг программы

```
Document.cs:
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.Entities
{
    public class Document
    {
        [Key]
        public int Id { get; set; }

        [Required]
        [MaxLength(500)]
        public string Title { get; set; } = string.Empty;

        [MaxLength(4000)]
        public string? Description { get; set; }

        [MaxLength(1000)]
        public string? SourceUrl { get; set; }

        [MaxLength(50)]
        public string SourceType { get; set; } = "manual"; // manual, import, api

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

        public int ViewCount { get; set; } = 0;

        public DateTime? LastSearchedAt { get; set; }

        public bool IsActive { get; set; } = true;

        [MaxLength(4000)]
        public string? Metadata { get; set; } // JSON

        // Навигационные свойства
        public ICollection<Paragraph> Paragraphs { get; set; } = new List<Paragraph>();
    }
}

Paragraph.cs:
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
```



```

using System.ComponentModel.DataAnnotations.Schema;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.Entities
{
    public class Paragraph
    {
        [Key]
        public int Id { get; set; }

        public int DocumentId { get; set; }

        [ForeignKey(nameof(DocumentId))]
        public Document? Document { get; set; }

        [Required]
        public string Content { get; set; } = string.Empty;

        public int ParagraphOrder { get; set; }

        [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
        public int WordCount { get; set; }

        [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
        public int CharCount { get; set; }

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        public DateTime? IndexedAt { get; set; } // Когда сгенерирован вектор

        public int VectorVersion { get; set; } = 1; // Версия модели эмбединга

        // Навигационные свойства
        public ParagraphVector? Vector { get; set; }

        // Не сохраняется в БД
        [NotMapped]
        public List<string>? ProcessedTokens { get; set; }

        [NotMapped]
        public float[]? Embedding { get; set; } // Вектор в памяти
    }
}

```

```

ParagraphVector.cs:
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.Entities
{

```

```

public class ParagraphVector
{
    [Key]
    public int Id { get; set; }

    public int ParagraphId { get; set; }

    [ForeignKey(nameof(ParagraphId))]
    public Paragraph? Paragraph { get; set; }

    [Required]
    public byte[] VectorData { get; set; } = Array.Empty<byte>(); // Binary хранение

    public int VectorDimension { get; set; } = 384;

    [MaxLength(100)]
    public string ModelName { get; set; } = "paraphrase-multilingual-MiniLM-L12-v2";

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public bool Normalized { get; set; } = true;
}
}

```

SearchQueryLog.cs:

```

using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.Entities
{
    public class SearchQueryLog
    {
        [Key]
        public int Id { get; set; }

        [Required]
        [MaxLength(500)]
        public string QueryText { get; set; } = string.Empty;

        [MaxLength(50)]
        public string? AlgorithmUsed { get; set; }

        public int ResultCount { get; set; }

        public int ExecutionTimeMs { get; set; }

        [MaxLength(100)]
        public string? UserSessionId { get; set; }

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        public int? FeedbackScore { get; set; } // 1-5
    }
}

```

```
}
```

StopWord.cs:

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

namespace SemanticSearch.Core.Entities

```
{
    public class StopWord
    {
        [Key]
        public int Id { get; set; }

        [Required]
        [MaxLength(100)]
        public string Word { get; set; } = string.Empty;

        [MaxLength(10)]
        public string Language { get; set; } = "en";

        public bool IsActive { get; set; } = true;
    }
}
```

Synonym.cs:

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

namespace SemanticSearch.Core.Entities

```
{
    public class Synonym
    {
        [Key]
        public int Id { get; set; }

        [Required]
        [MaxLength(100)]
        public string SourceWord { get; set; } = string.Empty;

        [Required]
        [MaxLength(100)]
        public string TargetWord { get; set; } = string.Empty;

        public decimal SimilarityScore { get; set; } = 1.00m;

        [MaxLength(50)]
        public string Source { get; set; } = "manual"; // manual, datamuse, api

        [MaxLength(10)]
```

```

        public string Language { get; set; } = "en";

        public bool IsActive { get; set; } = true;
    }
}

```

SearchAlgorithm.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.Enums
{
    public enum SearchAlgorithm
    {
        /// <summary>
        /// Гибридный: 80% вектор + 20% ключевые слова
        /// </summary>
        Hybrid = 0,

        /// <summary>
        /// Чисто векторный: 100% семантика
        /// </summary>
        Vector = 1
    }
}

```

DocumentDto.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.DTO
{
    public class DocumentDto
    {
        public int Id { get; set; }

        public string Title { get; set; } = string.Empty;

        public string? Description { get; set; }

        public string? SourceUrl { get; set; }

        public string SourceType { get; set; } = "manual";

        public DateTime CreatedAt { get; set; }

        public int ParagraphCount { get; set; }

        public bool IsIndexed { get; set; }
    }
}

```

```

        public List<ParagraphDto> Paragraphs { get; set; } = new List<ParagraphDto>();
    }
}

```

DocumentStatsDto.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.DTO
{
    public class DocumentStatsDto
    {
        public int TotalParagraphs { get; set; }
        public int IndexedParagraphs { get; set; }
        public int TotalWords { get; set; }
        public int TotalChars { get; set; }
        public DateTime? LastIndexedAt { get; set; }
    }
}

```

IndexingStatsDto.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.DTO
{
    public class IndexingStatsDto
    {
        public int TotalParagraphs { get; set; }
        public int IndexedParagraphs { get; set; }
        public int PendingParagraphs { get; set; }
        public string ModelName { get; set; } = string.Empty;
        public int VectorDimension { get; set; }
    }
}

```

ParagraphDto.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.DTO
{
    public class ParagraphDto
    {

```

```

    public int Id { get; set; }

    public int DocumentId { get; set; }

    public string Content { get; set; } = string.Empty;

    public int ParagraphOrder { get; set; }

    public int WordCount { get; set; }

    public DateTime? IndexedAt { get; set; }

    public bool HasVector { get; set; }
}

```

SearchMatchDto.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.DTO
{
    public class SearchMatchDto
    {
        public int ParagraphId { get; set; }

        public int DocumentId { get; set; }

        public string DocumentTitle { get; set; } = string.Empty;

        public string ParagraphContent { get; set; } = string.Empty;

        public double RelevanceScore { get; set; }

        public double VectorScore { get; set; }

        public double KeywordScore { get; set; }

        public int Rank { get; set; }

        public List<string> HighlightedWords { get; set; } = new List<string>();

        public Dictionary<string, double>? ScoreBreakdown { get; set; }
    }
}

```

SearchRequestDto.cs:

```

using SemanticSearch.Core.Enums;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

```

```

using System.Threading.Tasks;

namespace SemanticSearch.Core.DTO
{
    public class SearchRequestDto
    {
        public string Query { get; set; } = string.Empty;

        public SearchAlgorithm Algorithm { get; set; } = SearchAlgorithm.Hybrid;

        public int TopK { get; set; } = 10;

        public double? MinScore { get; set; }

        public bool UseCache { get; set; } = true;

        public bool LogQuery { get; set; } = true;
    }
}

```

SearchResponseDto.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Core.DTO
{
    public class SearchResponseDto
    {
        public string Query { get; set; } = string.Empty;

        public string AlgorithmUsed { get; set; } = string.Empty;

        public List<SearchMatchDto> Matches { get; set; } = new List<SearchMatchDto>();

        public int TotalTimeMs { get; set; }

        public int VectorSearchTimeMs { get; set; }

        public int KeywordSearchTimeMs { get; set; }

        public int TotalResults { get; set; }

        public bool FromCache { get; set; }
    }
}

```

AppDbContext.cs:

```

using Microsoft.EntityFrameworkCore;
using SemanticSearch.Core.Entities;
using System;
using System.Collections.Generic;
using System.Linq;

```

```

using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Infrastructure.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
        public AppDbContext() : base() { } // for testing
        public DbSet<Document> Documents { get; set; }
        public DbSet<Paragraph> Paragraphs { get; set; }
        public DbSet<ParagraphVector> ParagraphVectors { get; set; }
        public DbSet<StopWord> StopWords { get; set; }
        public DbSet<Synonym> Synonyms { get; set; }
        public DbSet<SearchQueryLog> SearchQueryLogs { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Document configuration
            modelBuilder.Entity<Document>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Title).IsRequired().HasMaxLength(500);
                entity.Property(e => e.SourceType).HasMaxLength(50).HasDefaultValue("manual");
                entity.HasIndex(e => e.IsActive);
                entity.HasIndex(e => e.CreatedAt);
            });

            // Paragraph configuration
            modelBuilder.Entity<Paragraph>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Content).IsRequired();
                entity.Property(e => e.WordCount).HasComputedColumnSql("LEN(Content) - LEN(REPLACE(Content, ' ', '')) + 1");
                entity.Property(e => e.CharCount).HasComputedColumnSql("LEN(Content)");
                entity.HasOne(e => e.Document)
                    .WithMany(d => d.Paragraphs)
                    .HasForeignKey(e => e.DocumentId)
                    .OnDelete(DeleteBehavior.Cascade);
                entity.HasIndex(e => e.DocumentId);
                entity.HasIndex(e => e.IndexedAt).HasFilter("[IndexedAt] IS NOT NULL");
            });

            // ParagraphVector configuration
            modelBuilder.Entity<ParagraphVector>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.HasIndex(e => e.ParagraphId).IsUnique();
                entity.Property(e => e.ModelName).HasMaxLength(100).HasDefaultValue("paraphrase-multilingual-MiniLM-L12-v2");
                entity.HasOne(e => e.Paragraph)
                    .WithOne(p => p.Vector)
                    .HasForeignKey<ParagraphVector>(e => e.ParagraphId)
                    .OnDelete(DeleteBehavior.Cascade);
            });
        }
    }
}

```



```

});

// StopWord configuration
modelBuilder.Entity<StopWord>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.HasIndex(e => e.Word).IsUnique();
    entity.Property(e => e.Language).HasMaxLength(10).HasDefaultValue("ru");
});

// Synonym configuration
modelBuilder.Entity<Synonym>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.HasIndex(e => e.SourceWord);
    entity.Property(e => e.Source).HasMaxLength(50).HasDefaultValue("manual");
    entity.Property(e => e.Language).HasMaxLength(10).HasDefaultValue("ru");
});

// SearchQueryLog configuration
modelBuilder.Entity<SearchQueryLog>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.Property(e => e.QueryText).IsRequired().HasMaxLength(500);
    entity.Property(e => e.AlgorithmUsed).HasMaxLength(50);
    entity.HasIndex(e => e.CreatedAt);
    entity.HasIndex(e => e.QueryText);
});

modelBuilder.Entity<Synonym>(entity =>
{
    entity.Property(e => e.SimilarityScore)
        .HasColumnType("decimal(3,2)");
});
}

public override Task<int> SaveChangesAsync(CancellationToken cancellationToken = default)
{
    // Автоматическое обновление UpdatedAt
    foreach (var entry in ChangeTracker.Entries<Document>())
    {
        if (entry.State == EntityState.Modified)
        {
            {
                entry.Entity.UpdatedAt = DateTime.UtcNow;
            }
        }
    }

    return base.SaveChangesAsync(cancellationToken);
}
}
}

```

VectorMath.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;

```

```

using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Helpers
{
    public static class VectorMath
    {
        /// <summary>
        /// Косинусное сходство между двумя векторами
        /// </summary>
        public static float CosineSimilarity(float[] a, float[] b)
        {
            if (a.Length != b.Length)
                throw new ArgumentException("Vectors must have the same dimension");

            float dotProduct = 0;
            float normA = 0;
            float normB = 0;

            for (int i = 0; i < a.Length; i++)
            {
                dotProduct += a[i] * b[i];
                normA += a[i] * a[i];
                normB += b[i] * b[i];
            }

            if (normA == 0 || normB == 0)
                return 0;

            return dotProduct / (MathF.Sqrt(normA) * MathF.Sqrt(normB));
        }

        /// <summary>
        /// Нормализация вектора (L2 норма)
        /// </summary>
        public static float[] Normalize(float[] vector)
        {
            float norm = 0;
            for (int i = 0; i < vector.Length; i++)
            {
                norm += vector[i] * vector[i];
            }

            norm = MathF.Sqrt(norm);
            if (norm == 0)
                return vector;

            var normalized = new float[vector.Length];
            for (int i = 0; i < vector.Length; i++)
            {
                normalized[i] = vector[i] / norm;
            }

            return normalized;
        }

        /// <summary>

```

```

/// Конвертация byte[] в float[] (безопасная версия)
/// </summary>
public static float[] BytesToFloats(byte[] bytes)
{
    if (bytes == null || bytes.Length == 0)
        return Array.Empty<float>();

    if (bytes.Length % 4 != 0)
        throw new ArgumentException($"Byte array length must be multiple of 4, got {bytes.Length}");

    var floats = new float[bytes.Length / 4];

    // Безопасная конвертация через BitConverter
    for (int i = 0; i < floats.Length; i++)
    {
        floats[i] = BitConverter.ToSingle(bytes, i * 4);
    }

    return floats;
}

/// <summary>
/// Конвертация float[] в byte[] (согласованная версия)
/// </summary>
public static byte[] FloatsToBytes(float[] floats)
{
    if (floats == null || floats.Length == 0)
        return Array.Empty<byte>();

    var bytes = new byte[floats.Length * 4];

    for (int i = 0; i < floats.Length; i++)
    {
        Buffer.BlockCopy(floats, i * 4, bytes, i * 4, 4);
    }

    return bytes;
}

/// <summary>
/// Средний вектор из множества векторов
/// </summary>
public static float[] Average(float[][] vectors)
{
    if (vectors.Length == 0)
        throw new ArgumentException("At least one vector required");

    var dimension = vectors[0].Length;
    var average = new float[dimension];

    for (int i = 0; i < vectors.Length; i++)
    {
        for (int j = 0; j < dimension; j++)
        {
            average[j] += vectors[i][j];
        }
    }
}

```

```

        for (int j = 0; j < dimension; j++)
        {
            average[j] /= vectors.Length;
        }

        return Normalize(average);
    }
}

```

DocumentRepository.cs:

```

using Microsoft.EntityFrameworkCore;
using SemanticSearch.Core.Entities;
using SemanticSearch.Infrastructure.Data;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Infrastructure.Repositories
{
    public class DocumentRepository : EfRepository<Document>
    {
        public DocumentRepository(AppDbContext context) : base(context) { }

        public async Task<IEnumerable<Document>> GetAllWithParagraphsAsync()
        {
            return await _dbSet
                .Include(d => d.Paragraphs)
                .Where(d => d.IsActive)
                .ToListAsync();
        }

        public async Task<Document?> GetByIdWithParagraphsAsync(int id)
        {
            return await _dbSet
                .Include(d => d.Paragraphs)
                .FirstOrDefaultAsync(d => d.Id == id);
        }

        public async Task IncrementViewCountAsync(int id)
        {
            {
                var document = await _dbSet.FindAsync(id);
                if (document != null)
                {
                    document.ViewCount++;
                    document.LastSearchedAt = System.DateTime.UtcNow;
                    await _context.SaveChangesAsync();
                }
            }
        }
    }
}

```

EfRepository.cs:

```

using Microsoft.EntityFrameworkCore;
using SemanticSearch.Infrastructure.Data;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Linq.Expressions;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Infrastructure.Repositories
{
    public class EfRepository<T> : IRepository<T> where T : class
    {
        protected readonly AppDbContext _context;
        protected readonly DbSet<T> _dbSet;

        public EfRepository(AppDbContext context)
        {
            _context = context;
            _dbSet = context.Set<T>();
        }

        public virtual async Task<T?> GetByIdAsync(int id)
        {
            return await _dbSet.FindAsync(id);
        }

        public virtual async Task<IEnumerable<T>> GetAllAsync()
        {
            return await _dbSet.ToListAsync();
        }

        public virtual async Task<IEnumerable<T>> FindAsync(Expression<Func<T, bool>> predicate)
        {
            return await _dbSet.Where(predicate).ToListAsync();
        }

        public virtual async Task<T> AddAsync(T entity)
        {
            await _dbSet.AddAsync(entity);
            await _context.SaveChangesAsync();
            return entity;
        }

        public virtual async Task<IEnumerable<T>> AddRangeAsync(IEnumerable<T> entities)
        {
            await _dbSet.AddRangeAsync(entities);
            await _context.SaveChangesAsync();
            return entities;
        }

        public virtual void Update(T entity)
        {
            _dbSet.Update(entity);
            _context.SaveChanges();
        }
    }
}

```

```

public virtual void Remove(T entity)
{
    _dbSet.Remove(entity);
    _context.SaveChanges();
}

public virtual void RemoveRange(IEnumerable<T> entities)
{
    _dbSet.RemoveRange(entities);
    _context.SaveChanges();
}

public virtual async Task<int> CountAsync()
{
    return await _dbSet.CountAsync();
}

public virtual async Task<int> CountAsync(Expression<Func<T, bool>> predicate)
{
    return await _dbSet.CountAsync(predicate);
}

public virtual async Task<bool> AnyAsync(Expression<Func<T, bool>> predicate)
{
    return await _dbSet.AnyAsync(predicate);
}
}
}

```

IRepository.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Linq.Expressions;
using System.Text;
using System.Threading.Tasks;

```

namespace SemanticSearch.Infrastructure.Repositories

```

{
    public interface IRepository<T> where T : class
    {
        Task<T?> GetByIdAsync(int id);
        Task<IEnumerable<T>> GetAllAsync();
        Task<IEnumerable<T>> FindAsync(Expression<Func<T, bool>> predicate);
        Task<T> AddAsync(T entity);
        Task<IEnumerable<T>> AddRangeAsync(IEnumerable<T> entities);
        void Update(T entity);
        void Remove(T entity);
        void RemoveRange(IEnumerable<T> entities);
        Task<int> CountAsync();
        Task<int> CountAsync(Expression<Func<T, bool>> predicate);
        Task<bool> AnyAsync(Expression<Func<T, bool>> predicate);
    }
}

```

LinguisticRepository.cs:

```

using Microsoft.EntityFrameworkCore;
using SemanticSearch.Core.Entities;
using SemanticSearch.Infrastructure.Data;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Infrastructure.Repositories
{
    public class LinguisticRepository
    {
        private readonly AppDbContext _context;

        public LinguisticRepository(AppDbContext context)
        {
            _context = context;
        }

        public async Task<List<StopWord>> GetStopWordsAsync()
        {
            return await _context.StopWords.ToListAsync();
        }

        public async Task<List<Synonym>> GetSynonymsAsync()
        {
            return await _context.Synonyms.ToListAsync();
        }

        public async Task<List<Paragraph>> GetAllParagraphsAsync()
        {
            return await _context.Paragraphs
                .Include(p => p.Document)
                .ToListAsync();
        }
    }
}

```

ParagraphRepository.cs:

```

using Microsoft.EntityFrameworkCore;
using SemanticSearch.Core.Entities;
using SemanticSearch.Infrastructure.Data;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Infrastructure.Repositories
{
    public class ParagraphRepository : EfRepository<Paragraph>
    {
        public ParagraphRepository(AppDbContext context) : base(context) { }

        public async Task<IEnumerable<Paragraph>> GetAllWithVectorsAsync()

```

```

    {
        return await _dbSet
            .Include(p => p.Document)
            .Include(p => p.Vector)
            .Where(p => p.Document.IsActive)
            .ToListAsync();
    }

    public async Task<IEnumerable<Paragraph>> GetNotIndexedAsync()
    {
        return await _dbSet
            .Include(p => p.Document)
            .Where(p => p.IndexedAt == null)
            .ToListAsync();
    }

    public async Task MarkAsIndexedAsync(int paragraphId)
    {
        var paragraph = await _dbSet.FindAsync(paragraphId);
        if (paragraph != null)
        {
            paragraph.IndexedAt = System.DateTime.UtcNow;
            await _context.SaveChangesAsync();
        }
    }
}

```

VectorRepository.cs:

```

using Microsoft.EntityFrameworkCore;
using SemanticSearch.Core.Entities;
using SemanticSearch.Infrastructure.Data;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace SemanticSearch.Infrastructure.Repositories
{
    public class VectorRepository : EfRepository<ParagraphVector>
    {
        public VectorRepository(AppDbContext context) : base(context) { }

        public async Task<ParagraphVector?> GetByParagraphIdAsync(int paragraphId)
        {
            return await _dbSet
                .Include(v => v.Paragraph)
                .FirstOrDefaultAsync(v => v.ParagraphId == paragraphId);
        }

        public async Task<IEnumerable<ParagraphVector>> GetAllWithParagraphsAsync()
        {
            return await _dbSet
                .Include(v => v.Paragraph)
                .ThenInclude(p => p.Document)
                .ToListAsync();
        }
    }
}

```



```

    }

    public async Task<int> GetIndexedCountAsync()
    {
        return await _dbSet.CountAsync();
    }
}

```

InMemoryVectorStore.cs:

```

using SemanticSearch.Application.Helpers;
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace SemanticSearch.Infrastructure.VectorStore
{
    public class InMemoryVectorStore : IVectorStore
    {
        private readonly ConcurrentDictionary<int, float[]> _vectors = new();
        private int _dimension = 384;
        private readonly SemaphoreSlim _lock = new(1, 1);

        public Task InitializeAsync(int dimension)
        {
            _dimension = dimension;
            return Task.CompletedTask;
        }

        public Task AddVectorAsync(int paragraphId, float[] vector)
        {
            if (vector.Length != _dimension)
                throw new ArgumentException($"Vector dimension must be {_dimension}");

            // Нормализуем вектор для косинусного сходства
            var normalized = VectorMath.Normalize(vector);
            _vectors[paragraphId] = normalized;
            return Task.CompletedTask;
        }

        public async Task AddVectorsAsync(IEnumerable<(int ParagraphId, float[] Vector)> vectors)
        {
            foreach (var (paragraphId, vector) in vectors)
            {
                await AddVectorAsync(paragraphId, vector);
            }
        }

        public Task<List<(int ParagraphId, float Score)>> SearchSimilarAsync(float[] queryVector, int topK)
        {
            if (_vectors.IsEmpty)
                return Task.FromResult(new List<(int, float)>());

            // Нормализуем запрос

```

```

        var normalizedQuery = VectorMath.Normalize(queryVector);

        // Параллельный расчёт косинусного сходства
        var results = _vectors
            .AsParallel()
            .Select(kvp => (
                ParagraphId: kvp.Key,
                Score: VectorMath.CosineSimilarity(normalizedQuery, kvp.Value)
            ))
            .OrderByDescending(r => r.Score)
            .Take(topK)
            .ToList();

        return Task.FromResult(results);
    }

    public Task RemoveVectorAsync(int paragraphId)
    {
        _vectors.TryRemove(paragraphId, out _);
        return Task.CompletedTask;
    }

    public Task<bool> ContainsAsync(int paragraphId)
    {
        return Task.FromResult(_vectors.ContainsKey(paragraphId));
    }

    public Task<int> CountAsync()
    {
        return Task.FromResult(_vectors.Count);
    }

    public Task ClearAsync()
    {
        _vectors.Clear();
        return Task.CompletedTask;
    }
}
}

```

IVectorStore.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Infrastructure.VectorStore
{
    public interface IVectorStore
    {
        Task InitializeAsync(int dimension);
        Task AddVectorAsync(int paragraphId, float[] vector);
        Task AddVectorsAsync(IEnumerable<(int ParagraphId, float[] Vector)> vectors);
        Task<List<(int ParagraphId, float Score)>> SearchSimilarAsync(float[] queryVector, int topK);
        Task RemoveVectorAsync(int paragraphId);
    }
}

```

```

        Task<bool> ContainsAsync(int paragraphId);
        Task<int> CountAsync();
        Task ClearAsync();
    }
}

```

ParagraphScore.cs:

```

using SemanticSearch.Core.Entities;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

namespace SemanticSearch.Infrastructure.AdditionalClasses

```

{
    public class ParagraphScore
    {
        public Paragraph Paragraph { get; set; } = null!;
        public double TotalScore { get; set; }
        public double TfIdfScore { get; set; }
        public double BM25Score { get; set; }
        public double VectorScore { get; set; }
        public List<string> MatchedTerms { get; set; } = new();
    }
}

```

SynonymResult.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

namespace SemanticSearch.Infrastructure.AdditionalClasses

```

{
    public class SynonymResult
    {
        public string Word { get; set; } = string.Empty;
        public double SimilarityScore { get; set; }
        public string Source { get; set; } = string.Empty; // "datamuse", "api", etc.
    }
}

```

IDocumentService.cs:

```

using SemanticSearch.Core.DTO;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

namespace SemanticSearch.Application.Interfaces

```

{
    public interface IDocumentService
    {

```

```

// CRUD для документов
Task<DocumentDto> CreateDocumentAsync(DocumentDto dto);
Task<DocumentDto?> GetDocumentAsync(int id);
Task<IEnumerable<DocumentDto>> GetAllDocumentsAsync(bool onlyActive = true);
Task<DocumentDto> UpdateDocumentAsync(int id, DocumentDto dto);
Task<bool> DeleteDocumentAsync(int id);

// CRUD для абзацев
Task<ParagraphDto> AddParagraphAsync(int documentId, string content, int order);
Task<ParagraphDto?> GetParagraphAsync(int id);
Task<IEnumerable<ParagraphDto>> GetParagraphsByDocumentAsync(int documentId);
Task<bool> UpdateParagraphAsync(int id, string content);
Task<bool> DeleteParagraphAsync(int id);

// Массовые операции
Task<int> ImportFromTextAsync(int documentId, string fullText, int maxParagraphLength = 500);
Task<int> BulkIndexParagraphsAsync(IEnumerable<int> paragraphIds);

// Статистика
Task<DocumentStatsDto> GetDocumentStatsAsync(int documentId);
}
}

```

IEmbeddingService.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Interfaces
{
    public interface IEmbeddingService
    {
        /// <summary>
        /// Инициализация модели (загрузка в память)
        /// </summary>
        Task InitializeAsync();

        /// <summary>
        /// Генерация эмбединга для одного текста
        /// </summary>
        Task<float[]> GenerateEmbeddingAsync(string text);

        /// <summary>
        /// Генерация эмбедингов для множества текстов (пакетно)
        /// </summary>
        Task<float[][]> GenerateEmbeddingsAsync(string[] texts);

        /// <summary>
        /// Размерность вектора модели
        /// </summary>
        int VectorDimension { get; }

        /// <summary>
        /// Название модели

```

```

    /// </summary>
    string ModelName { get; }

    /// <summary>
    /// Готова ли модель к работе
    /// </summary>
    bool IsReady { get; }
}

ILinguisticService.cs:
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Interfaces
{
    public interface ILinguisticService
    {
        /// <summary>
        /// Загрузка стоп-слов и локальных синонимов из БД
        /// </summary>
        Task LoadDataAsync();

        /// <summary>
        /// Токенизация текста с фильтрацией стоп-слов
        /// </summary>
        List<string> Tokenize(string text);

        /// <summary>
        /// Проверка: является ли слово стоп-словом
        /// </summary>
        bool IsStopWord(string word);

        /// <summary>
        /// Нормализация слова (лемматизация + lower)
        /// </summary>
        string NormalizeWord(string word);

        /// <summary>
        /// Расширение запроса синонимами (БД + API)
        /// </summary>
        Task<List<string>> ExpandQueryAsync(List<string> tokens);

        /// <summary>
        /// Получить только значимые токены (длина > 3, не стоп-слова)
        /// </summary>
        List<string> GetSignificantTokens(List<string> tokens);

        /// <summary>
        /// Очистить кэш лемматизации (при обновлении правил)
        /// </summary>
        void ClearCache();
    }
}

```

```

IRankingService.cs:
using SemanticSearch.Core.Entities;
using SemanticSearch.Infrastructure.AdditionalClasses;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Interfaces
{
    public interface IRankingService
    {
        /// <summary>
        /// Рассчитать релевантность с использованием всех методов
        /// </summary>
        Task<List<ParagraphScore>> RankAsync(
            string query,
            List<Paragraph> paragraphs,
            float[] queryVector,
            Core.Enums.SearchAlgorithm algorithm);

        /// <summary>
        /// Только BM25 скоринг
        /// </summary>
        Dictionary<int, double> CalculateBM25Scores(string query, List<Paragraph> paragraphs);

        /// <summary>
        /// Только TF-IDF скоринг
        /// </summary>
        Dictionary<int, double> CalculateTfidfScores(string query, List<Paragraph> paragraphs);

        /// <summary>
        /// Векторный скоринг (косинусное сходство)
        /// </summary>
        Dictionary<int, double> CalculateVectorScores(float[] queryVector, List<Paragraph> paragraphs);
    }
}

```

ISemanticSearchService.cs:

```

using SemanticSearch.Core.DTO;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Interfaces
{
    public interface ISemanticSearchService
    {
        /// <summary>
        /// Семантический поиск с выбором алгоритма
        /// </summary>
        Task<SearchResponseDto> SearchAsync(SearchRequestDto request);
    }
}

```

```

    /// <summary>
    /// Индексировать все неиндексированные абзацы
    /// </summary>
    Task<int> IndexPendingParagraphsAsync();

    /// <summary>
    /// Переиндексировать конкретный абзац
    /// </summary>
    Task<bool> ReindexParagraphAsync(int paragraphId);

    /// <summary>
    /// Получить статистику индексации
    /// </summary>
    Task<IndexingStatsDto> GetIndexingStatsAsync();
}
}

```

ISynonymApiService.cs:

```

using SemanticSearch.Infrastructure.AdditionalClasses;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Interfaces
{
    public interface ISynonymApiService
    {
        /// <summary>
        /// Получить синонимы из внешнего API
        /// </summary>
        Task<List<SynonymResult>> GetSynonymsAsync(string word, int maxResults = 10);

        /// <summary>
        /// Получить синонимы с кэшированием
        /// </summary>
        Task<List<SynonymResult>> GetSynonymsWithCacheAsync(string word, int maxResults = 10);
    }
}

```

DocumentService.cs:

```

using Microsoft.Extensions.Logging;
using SemanticSearch.Application.Interfaces;
using SemanticSearch.Core.DTO;
using SemanticSearch.Core.Entities;
using SemanticSearch.Infrastructure.Repositories;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Services
{
    public class DocumentService : IDocumentService

```

```

{
    private readonly DocumentRepository _docRepo;
    private readonly ParagraphRepository _paraRepo;
    private readonly ISemanticSearchService _searchService;
    private readonly ILogger<DocumentService> _logger;

    public DocumentService(
        DocumentRepository docRepo,
        ParagraphRepository paraRepo,
        ISemanticSearchService searchService,
        ILogger<DocumentService> logger)
    {
        _docRepo = docRepo;
        _paraRepo = paraRepo;
        _searchService = searchService;
        _logger = logger;
    }

    public async Task<DocumentDto> CreateDocumentAsync(DocumentDto dto)
    {
        var doc = new Document
        {
            Title = dto.Title,
            Description = dto.Description,
            SourceUrl = dto.SourceUrl,
            SourceType = dto.SourceType,
            Metadata = dto.Paragraphs.Any() ? $"{{\"initial_paragraphs\":{dto.Paragraphs.Count}}}" : null
        };

        var created = await _docRepo.AddAsync(doc);
        return MapToDto(created);
    }

    public async Task<DocumentDto?> GetDocumentAsync(int id)
    {
        var doc = await _docRepo.GetByIdWithParagraphsAsync(id);
        return doc == null ? null : MapToDto(doc);
    }

    public async Task<IEnumerable<DocumentDto>> GetAllDocumentsAsync(bool onlyActive = true)
    {
        var docs = onlyActive
            ? await _docRepo.FindAsync(d => d.IsActive)
            : await _docRepo.GetAllAsync();

        return docs.Select(MapToDto);
    }

    public async Task<DocumentDto> UpdateDocumentAsync(int id, DocumentDto dto)
    {
        var doc = await _docRepo.GetByIdAsync(id);
        if (doc == null)
            throw new KeyNotFoundException($"Document {id} not found");

        doc.Title = dto.Title;
        doc.Description = dto.Description;
        doc.SourceUrl = dto.SourceUrl;
    }

```



```

        doc.UpdatedAt = DateTime.UtcNow;

        _docRepo.Update(doc);
        return MapToDto(doc);
    }

    public async Task<bool> DeleteDocumentAsync(int id)
    {
        var doc = await _docRepo.GetByIdAsync(id);
        if (doc == null)
            return false;

        doc.IsActive = false;
        doc.UpdatedAt = DateTime.UtcNow;
        _docRepo.Update(doc);
        return true;
    }

    public async Task<ParagraphDto> AddParagraphAsync(int documentId, string content, int order)
    {
        var doc = await _docRepo.GetByIdAsync(documentId);
        if (doc == null)
            throw new KeyNotFoundException($"Document {documentId} not found");

        var para = new Paragraph
        {
            DocumentId = documentId,
            Content = content,
            ParagraphOrder = order
        };

        var created = await _paraRepo.AddAsync(para);
        return MapToDto(created);
    }

    public async Task<ParagraphDto?> GetParagraphAsync(int id)
    {
        var para = await _paraRepo.GetByIdAsync(id);
        return para == null ? null : MapToDto(para);
    }

    public async Task<IEnumerable<ParagraphDto>> GetParagraphsByDocumentAsync(int documentId)
    {
        var paras = await _paraRepo.FindAsync(p => p.DocumentId == documentId);
        return paras.OrderBy(p => p.ParagraphOrder).Select(MapToDto);
    }

    public async Task<bool> UpdateParagraphAsync(int id, string content)
    {
        var para = await _paraRepo.GetByIdAsync(id);
        if (para == null)
            return false;

        para.Content = content;
        para.IndexedAt = null; // Требуется переиндексации
        _paraRepo.Update(para);
        return true;
    }

```

```

}

public async Task<bool> DeleteParagraphAsync(int id)
{
    var para = await _paraRepo.GetByIdAsync(id);
    if (para == null)
        return false;

    _paraRepo.Remove(para);
    return true;
}

public async Task<int> ImportFromTextAsync(int documentId, string fullText, int maxParagraphLength = 500)
{
    var doc = await _docRepo.GetByIdAsync(documentId);
    if (doc == null)
        throw new KeyNotFoundException($"Document {documentId} not found");

    // Разбиение текста на абзацы
    var paragraphs = SplitIntoParagraphs(fullText, maxParagraphLength);

    var entities = paragraphs.Select((content, idx) => new Paragraph
    {
        DocumentId = documentId,
        Content = content,
        ParagraphOrder = idx + 1
    }).ToList();

    await _paraRepo.AddRangeAsync(entities);
    _logger.LogInformation($"Imported {entities.Count} paragraphs for document {documentId}");

    return entities.Count;
}

public async Task<int> BulkIndexParagraphsAsync(IEnumerable<int> paragraphIds)
{
    var indexed = 0;

    foreach (var id in paragraphIds)
    {
        try
        {
            {
                var success = await _searchService.ReindexParagraphAsync(id);
                if (success) indexed++;
            }
            catch (Exception ex)
            {
                _logger.LogError(ex, $"Failed to index paragraph {id}");
            }
        }
    }

    return indexed;
}

public async Task<DocumentStatsDto> GetDocumentStatsAsync(int documentId)
{
    var paragraphs = await _paraRepo.FindAsync(p => p.DocumentId == documentId);

```

```

return new DocumentStatsDto
{
    TotalParagraphs = paragraphs.Count(),
    IndexedParagraphs = paragraphs.Count(p => p.IndexedAt != null),
    TotalWords = paragraphs.Sum(p => p.WordCount),
    TotalChars = paragraphs.Sum(p => p.CharCount),
    LastIndexedAt = paragraphs.Max(p => p.IndexedAt)
};
}

// Вспомогательные методы
private DocumentDto MapToDto(Document doc)
{
    return new DocumentDto
    {
        Id = doc.Id,
        Title = doc.Title,
        Description = doc.Description,
        SourceUrl = doc.SourceUrl,
        SourceType = doc.SourceType,
        CreatedAt = doc.CreatedAt,
        ParagraphCount = doc.Paragraphs?.Count ?? 0,
        IsIndexed = doc.Paragraphs?.All(p => p.IndexedAt != null) ?? false,
        Paragraphs = doc.Paragraphs?.Select(MapToDto).ToList() ?? new List<ParagraphDto>()
    };
}

private ParagraphDto MapToDto(Paragraph para)
{
    return new ParagraphDto
    {
        Id = para.Id,
        DocumentId = para.DocumentId,
        Content = para.Content,
        ParagraphOrder = para.ParagraphOrder,
        WordCount = para.WordCount,
        IndexedAt = para.IndexedAt,
        HasVector = para.IndexedAt != null
    };
}

private List<string> SplitIntoParagraphs(string text, int maxLength)
{
    var paragraphs = new List<string>();

    // Разбиваем по двойным переносам строки
    var raw = Regex.Split(text, @"\n\s*\n");

    foreach (var block in raw)
    {
        var clean = block.Trim();
        if (string.IsNullOrEmpty(clean))
            continue;

        // Если блок слишком длинный - разбиваем по предложениям
        if (clean.Length > maxLength)

```

```

    {
        var sentences = Regex.Split(clean, @"[.!?]+\s*");
        var current = "";

        foreach (var sentence in sentences)
        {
            if (string.IsNullOrEmpty(sentence))
                continue;

            if ((current + sentence).Length <= maxLength)
            {
                current += sentence + ". ";
            }
            else
            {
                if (!string.IsNullOrEmpty(current))
                    paragraphs.Add(current.Trim());
                current = sentence + ". ";
            }
        }
        if (!string.IsNullOrEmpty(current))
            paragraphs.Add(current.Trim());
    }
    else
    {
        paragraphs.Add(clean);
    }
}

return paragraphs;
}
}
}

```

EmbeddingService.cs:

```

using Microsoft.Extensions.Logging;
using Microsoft.ML.OnnxRuntime;
using Microsoft.ML.OnnxRuntime.Tensors;
using SemanticSearch.Application.Helpers;
using SemanticSearch.Application.Interfaces;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Services
{
    /// <summary>
    /// Сервис генерации эмбедингов через SBERT (ONNX)
    /// Работает без внешних токенизаторов - использует простой fallback
    /// </summary>
    public class EmbeddingService : IEmbeddingService, IDisposable
    {
        private readonly ILogger<EmbeddingService> _logger;
    }
}

```

```

private readonly string _modelPath;

private InferenceSession? _session;
private SimpleTokenizer? _tokenizer;
private bool _isInitialized;
private readonly SemaphoreSlim _initLock = new(1, 1);

public int VectorDimension => 384;
public string ModelName => "all-MiniLM-L6-v2";
public bool IsReady => _isInitialized && _session != null;

public EmbeddingService(ILogger<EmbeddingService> logger, string modelPath)
{
    _logger = logger;
    _modelPath = modelPath;
}

public async Task InitializeAsync()
{
    if (_isInitialized)
        return;

    await _initLock.WaitAsync();
    try
    {
        if (_isInitialized)
            return;

        _logger.LogInformation($"Loading SBERT model from {_modelPath}");

        if (!Directory.Exists(_modelPath))
            throw new DirectoryNotFoundException($"Model directory not found: {_modelPath}");

        // Поиск ONNX модели
        var onnxFile = FindModelFile();
        if (string.IsNullOrEmpty(onnxFile))
        {
            _logger.LogError("ONNX model not found. Please download model.onnx from HuggingFace");
            _logger.LogWarning($"URL: https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2/tree/main/onnx");
            return;
        }

        _logger.LogInformation($"Loading ONNX model: {onnxFile}");

        // Инициализация ONNX Runtime
        var sessionOptions = new SessionOptions
        {
            InterOpNumThreads = 1,
            IntraOpNumThreads = Math.Max(1, Environment.ProcessorCount / 2)
        };
        sessionOptions.AppendExecutionProvider_CPU(0);

        _session = new InferenceSession(onnxFile, sessionOptions);
        _logger.LogInformation("ONNX session created");

        // Инициализация простого токенизатора

```

```

        _tokenizer = new SimpleTokenizer(_modelPath);
        _logger.LogInformation("Tokenizer initialized");

        _isInitialized = true;
        _logger.LogInformation($"✓ Model ready. Dimension: {VectorDimension}");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Failed to initialize embedding model");
        throw;
    }
    finally
    {
        _initLock.Release();
    }
}

private string? FindModelFile()
{
    var candidates = new[]
    {
        Path.Combine(_modelPath, "onnx", "model.onnx"),
        Path.Combine(_modelPath, "model.onnx"),
        Path.Combine(_modelPath, "onnx", "model_quantized.onnx"),
        Path.Combine(_modelPath, "pytorch_model.bin")
    };

    return candidates.FirstOrDefault(File.Exists);
}

public async Task<float[]> GenerateEmbeddingAsync(string text)
{
    if (!IsReady)
        await InitializeAsync();

    if (string.IsNullOrEmpty(text))
        return new float[VectorDimension];

    var embeddings = await GenerateEmbeddingsAsync(new[] { text });
    return embeddings.FirstOrDefault() ?? new float[VectorDimension];
}

public async Task<float[][]> GenerateEmbeddingsAsync(string[] texts)
{
    if (!IsReady)
        await InitializeAsync();

    if (texts == null || texts.Length == 0)
        return Array.Empty<float[]>();

    var results = new float[texts.Length][];
    var batchSize = 4;

    for (int i = 0; i < texts.Length; i += batchSize)
    {
        var batch = texts.Skip(i).Take(batchSize).ToArray();
        var batchResults = await ProcessBatchAsync(batch);
    }
}

```

```

        for (int j = 0; j < batch.Length && i + j < results.Length; j++)
        {
            results[i + j] = batchResults[j];
        }
    }

    return results;
}

private async Task<float[][]> ProcessBatchAsync(string[] texts)
{
    _logger.LogInformation($"Processing batch on Thread {Thread.CurrentThread.ManagedThreadId}");
    return await Task.Run(() =>
    {
        if (_session == null || _tokenizer == null)
            throw new InvalidOperationException("Model or tokenizer not initialized");

        // Токенизация
        var encoded = _tokenizer.EncodeBatch(texts);

        // Создание тензоров с long
        var inputIdsTensor = CreateLongTensor(encoded.InputIds, encoded.MaxLength);
        var attentionMaskTensor = CreateLongTensor(encoded.AttentionMask, encoded.MaxLength);
        var tokenTypeIdsTensor = CreateLongTensor(encoded.TokenTypeIds, encoded.MaxLength);

        // Подготовка входов для ONNX
        var inputs = new List<NamedOnnxValue>
        {
            NamedOnnxValue.CreateFromTensor("input_ids", inputIdsTensor),
            NamedOnnxValue.CreateFromTensor("attention_mask", attentionMaskTensor),
            NamedOnnxValue.CreateFromTensor("token_type_ids", tokenTypeIdsTensor)
        };

        // Инференс
        using var results = _session.Run(inputs);

        // Извлечение эмбедингов
        var embeddings = ExtractEmbeddings(results, texts.Length);

        // Нормализация
        for (int i = 0; i < embeddings.Length; i++)
        {
            embeddings[i] = VectorMath.Normalize(embeddings[i]);
        }

        return embeddings;
    });
}

private DenseTensor<long> CreateLongTensor(int[][] values, int maxLength)
{
    var tensor = new DenseTensor<long>(new[] { values.Length, maxLength });

    for (int b = 0; b < values.Length; b++)
    {
        for (int i = 0; i < values[b].Length && i < maxLength; i++)
        {

```

```

        tensor[b, i] = values[b][i]; // Автоматическое преобразование int в long
    }
}

return tensor;
}

private float[][] ExtractEmbeddings(IDisposableReadOnlyCollection<DisposableNamedOnnxValue> results, int
batchSize)
{
    var output = results.FirstOrDefault(r =>
        r.Name == "sentence_embedding" ||
        r.Name == "last_hidden_state" ||
        r.Name == "token_embeddings");

    if (output == null)
    {
        foreach (var r in results)
        {
            try
            {
                var t = r.AsTensor<float>();
                if (t != null && t.Dimensions.Length >= 2)
                {
                    output = r;
                    break;
                }
            }
            catch { continue; }
        }
    }

    if (output == null)
        throw new InvalidOperationException("Could not find a valid output tensor in ONNX results.");

    // тензор и его размерности
    var tensor = output.AsTensor<float>();
    var dims = tensor.Dimensions; // Теперь это работает корректно

    // Обработка разных форматов выхода
    if (dims.Length == 2 && dims[0] == batchSize)
    {
        // [batch, hidden] - прямой эмбединг предложения
        return ExtractDirectEmbeddings(tensor, batchSize, (int)dims[1]);
    }
    else if (dims.Length == 3)
    {
        // [batch, seq, hidden] - нужен mean pooling
        return ExtractWithMeanPooling(tensor, batchSize, (int)dims[1], (int)dims[2]);
    }

    throw new InvalidOperationException($"Unexpected output shape: [{dims.ToString()}]");
}

private float[][] ExtractDirectEmbeddings(Tensor<float> tensor, int batchSize, int hiddenSize)
{
    var embeddings = new float[batchSize][];

```



```

    for (int b = 0; b < batchSize; b++)
    {
        embeddings[b] = new float[hiddenSize];
        for (int h = 0; h < hiddenSize; h++)
        {
            embeddings[b][h] = tensor[b, h];
        }
    }

    return embeddings;
}

private float[][] ExtractWithMeanPooling(Tensor<float> tensor, int batchSize, int seqLen, int hiddenSize)
{
    var embeddings = new float[batchSize][];

    for (int b = 0; b < batchSize; b++)
    {
        embeddings[b] = new float[hiddenSize];
        int count = 0;

        for (int s = 0; s < seqLen; s++)
        {
            // Пропуск паддинг токенов (эвристика)
            var firstHidden = tensor[b, s, 0];
            if (firstHidden == 0 && s > 0)
                continue;

            for (int h = 0; h < hiddenSize; h++)
            {
                embeddings[b][h] += tensor[b, s, h];
            }
            count++;
        }

        // Усреднение
        if (count > 0)
        {
            for (int h = 0; h < hiddenSize; h++)
            {
                embeddings[b][h] /= count;
            }
        }
    }

    return embeddings;
}

public void Dispose()
{
    _session?.Dispose();
    _tokenizer?.Dispose();
    _initLock?.Dispose();
}
}

```

```

/// <summary>
/// Простой токенизатор для SBERT моделей
/// Не требует внешних зависимостей - работает на основе базовых правил
/// </summary>
public class SimpleTokenizer : IDisposable
{
    private readonly Dictionary<string, int> _vocab;
    private readonly int _maxSequenceLength;
    private readonly int _padTokenId;
    private readonly int _clsTokenId;
    private readonly int _sepTokenId;
    private readonly int _unkTokenId;

    public SimpleTokenizer(string modelPath, int maxSequenceLength = 128)
    {
        _maxSequenceLength = maxSequenceLength;
        _padTokenId = 0;
        _clsTokenId = 101; // [CLS] для BERT-подобных моделей
        _sepTokenId = 102; // [SEP]
        _unkTokenId = 100; // [UNK]

        // Попытка загрузить vocab из разных возможных файлов
        _vocab = LoadVocab(modelPath) ?? CreateMinimalVocab();
    }

    private Dictionary<string, int>? LoadVocab(string modelPath)
    {
        var vocabPaths = new[]
        {
            Path.Combine(modelPath, "vocab.txt"),
            Path.Combine(modelPath, "tokenizer.json"),
            Path.Combine(modelPath, "vocab.json")
        };

        foreach (var path in vocabPaths)
        {
            if (!File.Exists(path))
                continue;

            try
            {
                if (path.EndsWith(".txt"))
                    return LoadVocabTxt(path);
                else if (path.EndsWith(".json"))
                    return LoadVocabJson(path);
            }
            catch
            {
            }
        }

        return null;
    }

    private Dictionary<string, int> LoadVocabTxt(string path)
    {
        var vocab = new Dictionary<string, int>();
    }

```

```

var lines = File.ReadAllLines(path);

for (int i = 0; i < lines.Length && i < 30000; i++)
{
    var token = lines[i].Trim();
    if (!string.IsNullOrEmpty(token))
        vocab[token] = i;
}

return vocab;
}

private Dictionary<string, int> LoadVocabJson(string path)
{
    // Упрощённый парсер для vocab.json
    var vocab = new Dictionary<string, int>();
    var content = File.ReadAllText(path);

    // Простой regex для извлечения "токен": номер
    var matches = Regex.Matches(content, @"""([\^"]+)""\s*:\s*(\d+)");

    foreach (Match match in matches)
    {
        if (vocab.Count >= 30000) break;
        vocab[match.Groups[1].Value] = int.Parse(match.Groups[2].Value);
    }

    return vocab;
}

private Dictionary<string, int> CreateMinimalVocab()
{
    // Минимальный словарь для работы без vocab файла
    var vocab = new Dictionary<string, int>();

    // Special tokens
    vocab["[PAD]"] = 0;
    vocab["[UNK]"] = 100;
    vocab["[CLS]"] = 101;
    vocab["[SEP]"] = 102;
    vocab["[MASK]"] = 103;

    var commonRu = new[] { "а", "и", "в", "не", "на", "с", "к", "по", "что", "как",
        "это", "тот", "быть", "он", "она", "оно", "мы", "вы", "они",
        "токен", "текст", "слово", "модель", "нейрон", "сеть", "данные" };

    int id = 1000;
    foreach (var token in commonRu)
    {
        vocab[token] = id++;
        vocab[token.ToLower()] = id++;
    }

    return vocab;
}

public EncodedBatch EncodeBatch(string[] texts)

```

```

{
    var batchSize = texts.Length;
    var inputIds = new int[batchSize][];
    var attentionMask = new int[batchSize][];
    var tokenTypeIds = new int[batchSize][];
    int maxLength = 0;

    // Первый проход: токенизация и определение максимальной длины
    for (int b = 0; b < batchSize; b++)
    {
        var tokens = Tokenize(texts[b]);
        var ids = new List<int> { _clsTokenId }; // [CLS] в начале

        foreach (var token in tokens)
        {
            if (ids.Count >= _maxSequenceLength - 1) break;
            ids.Add(_vocab.TryGetValue(token, out var id) ? id : _unkTokenId);
        }

        if (ids.Count < _maxSequenceLength)
            ids.Add(_sepTokenId); // [SEP] в конце

        inputIds[b] = ids.ToArray();
        attentionMask[b] = new int[ids.Count];
        Array.Fill(attentionMask[b], 1);
        tokenTypeIds[b] = new int[ids.Count]; // Все 0 для SBERT

        maxLength = Math.Max(maxLength, ids.Count);
    }

    // Второй проход: паддинг до maxLength
    for (int b = 0; b < batchSize; b++)
    {
        if (inputIds[b].Length < maxLength)
        {
            Array.Resize(ref inputIds[b], maxLength);
            Array.Resize(ref attentionMask[b], maxLength);
            Array.Resize(ref tokenTypeIds[b], maxLength);

            // заполнение паддингом
            for (int i = inputIds[b].Length - (maxLength - inputIds[b].Where(x => x != 0).Count()); i < maxLength;
i++)
            {
                if (inputIds[b][i] == 0)
                {
                    inputIds[b][i] = _padTokenId;
                    attentionMask[b][i] = 0;
                }
            }
        }
    }

    return new EncodedBatch(inputIds, attentionMask, tokenTypeIds, maxLength);
}

private string[] Tokenize(string text)
{

```

```

        if (string.IsNullOrEmpty(text))
            return Array.Empty<string>();

        // Базовая токенизация: по пробелам и знакам препинания
        return Regex.Split(text.ToLowerInvariant(), @"[\s\p{P}]+")
            .Where(t => !string.IsNullOrEmpty(t))
            .Take(_maxSequenceLength - 2)
            .ToArray();
    }

    public void Dispose() { }
}

public class EncodedBatch
{
    public int[][] InputIds { get; }
    public int[][] AttentionMask { get; }
    public int[][] TokenTypeIds { get; }
    public int MaxLength { get; }

    public EncodedBatch(int[][] inputIds, int[][] attentionMask, int[][] tokenTypeIds, int maxLength)
    {
        InputIds = inputIds;
        AttentionMask = attentionMask;
        TokenTypeIds = tokenTypeIds;
        MaxLength = maxLength;
    }
}
}

```

LinguisticService.cs:

```

using Microsoft.Extensions.Caching.Memory;
using Microsoft.Extensions.Logging;
using SemanticSearch.Application.Interfaces;
using SemanticSearch.Infrastructure;
using SemanticSearch.Infrastructure.Repositories;
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

namespace SemanticSearch.Application.AdditionalClasses
{
    public class LinguisticService : ILinguisticService
    {
        private readonly LinguisticRepository _repo;
        private readonly ISynonymApiService _synonymApi;
        private readonly IMemoryCache _cache;
        private readonly ILogger<LinguisticService> _logger;

        private HashSet<string> _stopWords = new();
        private Dictionary<string, List<string>> _localSynonyms = new();
        private readonly ConcurrentDictionary<string, string> _lemmaCache = new();
    }
}

```

```

private static readonly string[] _russianSuffixes = new[]
{
    "ами", "ями", "ого", "его", "ому", "ему", "ыми", "ими",
    "ой", "ей", "ым", "им", "ую", "юю", "ое", "ее", "ых", "их",
    "ть", "ти", "чь", "ешь", "ëshь", "ет", "ёт", "ем", "ём",
    "ете", "ёте", "ут", "ют", "ат", "ят", "л", "ла", "ло", "ли",
    "ться", "лся", "лась", "лось", "ый", "ий", "ом", "ем",
    "ах", "ях", "а", "я", "о", "е", "у", "ю", "ы", "и", "ь"
};

public LinguisticService(
    LinguisticRepository repo,
    ISynonymApiService synonymApi,
    IMemoryCache cache,
    ILogger<LinguisticService> logger)
{
    _repo = repo;
    _synonymApi = synonymApi;
    _cache = cache;
    _logger = logger;
}

public async Task LoadDataAsync()
{
    var stops = await _repo.GetStopWordsAsync();
    _stopWords = new HashSet<string>(stops.Select(s => s.Word.ToLower()));
    _logger.LogInformation($"Loaded {_stopWords.Count} stop words");

    var syns = await _repo.GetSynonymsAsync();
    _localSynonyms = syns
        .Where(s => s.IsActive)
        .GroupBy(s => s.SourceWord.ToLower())
        .ToDictionary(g => g.Key, g => g.Select(x => x.TargetWord.ToLower()).ToList());
    _logger.LogInformation($"Loaded {_localSynonyms.Count} local synonym groups");
}

public List<string> Tokenize(string text)
{
    if (string.IsNullOrEmpty(text))
        return new List<string>();

    // Извлекаются слова (кириллица, латиница, цифры)
    var matches = Regex.Matches(text.ToLower(), @"[a-zA-Za-яA-Я0-9]+");

    // фильтр стоп-слов
    return matches
        .Select(m => m.Value)
        .Where(t => !IsStopWord(t))
        .ToList();
}

public bool IsStopWord(string word)
{
    return _stopWords.Contains(word.ToLower());
}

public string NormalizeWord(string word)

```

```

{
    var lower = word.ToLower();
    if (IsStopWord(lower))
        return string.Empty;

    if (lower.Length < 3)
        return lower;

    return _lemmaCache.GetOrAdd(lower, key =>
    {
        var nounSuffixes = new[]
        {
            "ами", "ями", "ого", "его", "ому", "ему",
            "ыми", "ими", "ах", "ях",
            "ой", "ей", "ым", "им",
            "ую", "юю", "ое", "её", "ых", "их"
        };

        foreach (var suffix in nounSuffixes.OrderByDescending(s => s.Length))
        {
            if (key.EndsWith(suffix) && key.Length > suffix.Length + 3)
                return key.Substring(0, key.Length - suffix.Length);
        }

        // Окончания глаголов
        var verbSuffixes = new[]
        {
            "ться", "тся", "ешь", "ёшь", "ет", "ёт",
            "ем", "ём", "ете", "ёте", "ут", "ют", "ат", "ят",
            "ла", "ло", "ли", "л"
        };

        foreach (var suffix in verbSuffixes.OrderByDescending(s => s.Length))
        {
            if (key.EndsWith(suffix) && key.Length > suffix.Length + 3)
            {
                var stem = key.Substring(0, key.Length - suffix.Length);
                return stem + "ть";
            }
        }

        return key;
    });
}

public async Task<List<string>> ExpandQueryAsync(List<string> tokens)
{
    var expanded = new HashSet<string>();

    foreach (var token in tokens)
    {
        var normalized = NormalizeWord(token);
        if (!string.IsNullOrEmpty(normalized))
        {
            expanded.Add(normalized);

            // Локальные синонимы из БД

```

```

        if (_localSynonyms.TryGetValue(normalized, out var localSyns))
        {
            foreach (var syn in localSyns)
                expanded.Add(syn);
        }

        // API синонимы (только для значимых слов)
        if (token.Length > 3 && normalized.Length > 3)
        {
            var apiSyns = await _synonymApi.GetSynonymsWithCacheAsync(normalized, 5);
            foreach (var syn in apiSyns)
                expanded.Add(syn.Word.ToLower());
        }

        if (token.Length > 3)
            expanded.Add(token.ToLower());
    }
}

_logger.LogDebug($"Expanded {tokens.Count} tokens to {expanded.Count} with synonyms");
return expanded.ToList();
}

public List<string> GetSignificantTokens(List<string> tokens)
{
    return tokens
        .Where(t => t.Length > 3 && !IsStopWord(t))
        .Select(NormalizeWord)
        .Where(t => !string.IsNullOrEmpty(t))
        .Distinct()
        .ToList();
}

public void ClearCache()
{
    _lemmaCache.Clear();
    _logger.LogInformation("Lemma cache cleared");
}
}
}

```

RankingService.cs:

```

using Microsoft.Extensions.Logging;
using SemanticSearch.Application.Helpers;
using SemanticSearch.Application.Interfaces;
using SemanticSearch.Core.Entities;
using SemanticSearch.Core.Enums;
using SemanticSearch.Infrastructure.AdditionalClasses;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Services
{
    public class RankingService : IRankingService

```



```

{
    private readonly ILinguisticService _linguistic;
    private readonly ILogger<RankingService> _logger;

    public RankingService(ILinguisticService linguistic, ILogger<RankingService> logger)
    {
        _linguistic = linguistic;
        _logger = logger;
    }

    public void InitializeStats(IEnumerable<Paragraph> paragraphs)
    {
    }

    public async Task<List<ParagraphScore>> RankAsync(string query, List<Paragraph> paragraphs, float[] queryVector, SearchAlgorithm algorithm)
    {
        _logger.LogInformation($"Ranking on Thread {Thread.CurrentThread.ManagedThreadId}, ManagedThreadId={Thread.CurrentThread.ManagedThreadId}");
        var results = new List<ParagraphScore>();

        // Веса для режимов
        double vectorWeight = algorithm == SearchAlgorithm.Hybrid ? 0.80 : 1.0;
        double keywordWeight = algorithm == SearchAlgorithm.Hybrid ? 0.20 : 0.0;

        // Подсчёт бонуса за ключевые слова (только для Hybrid)
        var keywordBonus = algorithm == SearchAlgorithm.Hybrid
            ? CalculateKeywordBonus(query, paragraphs)
            : new Dictionary<int, double>();

        foreach (var paragraph in paragraphs)
        {
            if (paragraph.Embedding == null || paragraph.Embedding.Length == 0)
                continue;

            // Векторный скор (0.0 - 1.0)
            var similarity = VectorMath.CosineSimilarity(queryVector, paragraph.Embedding);
            var vectorScore = (similarity + 1) / 2;

            // Ключевые слова (0.0 - 1.0)
            var keywordScore = keywordBonus.GetValueOrDefault(paragraph.Id, 0);

            // Итоговый скор
            var totalScore = (vectorScore * vectorWeight) + (keywordScore * keywordWeight);
            totalScore = Math.Min(totalScore, 1.0); // Кап на 100%

            results.Add(new ParagraphScore
            {
                Paragraph = paragraph,
                VectorScore = vectorScore,
                TfidfScore = keywordScore,
                BM25Score = 0,
                TotalScore = totalScore
            });
        }

        return results.OrderByDescending(r => r.TotalScore).ToList();
    }
}

```

```

    }

    // простой бонус за ключевые слова
    private Dictionary<int, double> CalculateKeywordBonus(string query, List<Paragraph> paragraphs)
    {
        var bonuses = new Dictionary<int, double>();
        var queryWords = query.Split(' ', StringSplitOptions.RemoveEmptyEntries)
            .Select(w => w.ToLower().Trim(new[] { '!', '?', ';', ':', '\t', '\n' }))
            .Where(w => w.Length > 3)
            .ToList();

        foreach (var p in paragraphs)
        {
            var content = p.Content.ToLower();
            var title = p.Document?.Title?.ToLower() ?? "";

            double bonus = 0;
            foreach (var word in queryWords)
            {
                if (title.Contains(word)) bonus += 0.5;
                else if (content.Contains(word)) bonus += 0.15;
            }

            bonuses[p.Id] = Math.Min(bonus, 1.0);
        }

        return bonuses;
    }

    public Dictionary<int, double> CalculateTfidfScores(string query, List<Paragraph> paragraphs)
    {
        return new Dictionary<int, double>();
    }

    public Dictionary<int, double> CalculateBM25Scores(string query, List<Paragraph> paragraphs)
    {
        return new Dictionary<int, double>();
    }

    public Dictionary<int, double> CalculateVectorScores(float[] queryVector, List<Paragraph> paragraphs)
    {
        return new Dictionary<int, double>();
    }
}

```

SemanticSearchService.cs:

```

using Microsoft.Extensions.Caching.Memory;
using Microsoft.Extensions.Logging;
using SemanticSearch.Application.Helpers;
using SemanticSearch.Application.Interfaces;
using SemanticSearch.Core.DTO;
using SemanticSearch.Core.Entities;
using SemanticSearch.Core.Enums;
using SemanticSearch.Infrastructure.Repositories;
using SemanticSearch.Infrastructure.VectorStore;

```

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Threading.Tasks;

namespace SemanticSearch.Application.Services
{
    public class SemanticSearchService : ISemanticSearchService
    {
        private readonly IEmbeddingService _embeddingService;
        private readonly IRankingService _rankingService;
        private readonly ILinguisticService _linguisticService;
        private readonly ParagraphRepository _paragraphRepo;
        private readonly VectorRepository _vectorRepo;
        private readonly IVectorStore _vectorStore;
        private readonly IMemoryCache _cache;
        private readonly ILogger<SemanticSearchService> _logger;

        private bool _isInitialized;
        private readonly SemaphoreSlim _initLock = new(1, 1);

        public SemanticSearchService(
            IEmbeddingService embeddingService,
            IRankingService rankingService,
            ILinguisticService linguisticService,
            ParagraphRepository paragraphRepo,
            VectorRepository vectorRepo,
            IVectorStore vectorStore,
            IMemoryCache cache,
            ILogger<SemanticSearchService> logger)
        {
            _embeddingService = embeddingService;
            _rankingService = rankingService;
            _linguisticService = linguisticService;
            _paragraphRepo = paragraphRepo;
            _vectorRepo = vectorRepo;
            _vectorStore = vectorStore;
            _cache = cache;
            _logger = logger;
        }

        public async Task<SearchResponseDto> SearchAsync(SearchRequestDto request)
        {
            var process = Process.GetCurrentProcess();
            _logger.LogInformation($"CPU Count: {Environment.ProcessorCount}");
            _logger.LogInformation($"Process Threads: {process.Threads.Count}");
            var stopwatch = Stopwatch.StartNew();
            var threadId = Thread.CurrentThread.ManagedThreadId;

            _logger?.LogInformation($"Search started on thread {threadId}: '{request.Query}'");
            var response = new SearchResponseDto
            {
                Query = request.Query,
                AlgorithmUsed = request.Algorithm.ToString()
            };
        }
    }
}

```

```

// Кэширование
if (request.UseCache)
{
    var cacheKey = $"search_{request.Algorithm}_{request.Query.ToLower().Trim()}";

    var cached = _cache.Get<SearchResponseDto>(cacheKey);
    if (cached != null)
    {
        cached.FromCache = true;
        return cached;
    }
}

// Инициализация при первом запуске
if (!_isInitialized)
{
    await _initLock.WaitAsync();
    try
    {
        if (!_isInitialized)
        {
            await InitializeAsync();
            _isInitialized = true;
        }
    }
    finally
    {
        _initLock.Release();
    }
}

// Генерация вектора запроса (для векторного поиска)
float[]? queryVector = null;
var vectorSearchTime = 0;

if (request.Algorithm is SearchAlgorithm.Vector or SearchAlgorithm.Hybrid)
{
    var vectorStopwatch = Stopwatch.StartNew();
    queryVector = await _embeddingService.GenerateEmbeddingAsync(request.Query);
    vectorSearchTime = (int)vectorStopwatch.ElapsedMilliseconds;
}

// Загрузка абзацев
var paragraphs = await _paragraphRepo.GetAllWithVectorsAsync();
var activeParagraphs = paragraphs
    .Where(p => p.Document?.IsActive == true)
    .ToList();

// Загрузка векторов в память для поиска
foreach (var p in activeParagraphs)
{
    if (p.Vector?.VectorData != null && p.Embedding == null)
    {
        p.Embedding = VectorMath.BytesToFloats(p.Vector.VectorData);
    }
}

```

```

// Ранжирование
var keywordSearchTime = 0;
var keywordStopwatch = Stopwatch.StartNew();

var ranked = await _rankingService.RankAsync(
    request.Query,
    activeParagraphs,
    queryVector ?? Array.Empty<float>(),
    request.Algorithm);

keywordSearchTime = (int)keywordStopwatch.ElapsedMilliseconds;

// Формирование результатов
response.Matches = ranked
    .Take(request.TopK)
    .Select((r, idx) => new SearchMatchDto
    {
        ParagraphId = r.Paragraph.Id,
        DocumentId = r.Paragraph.DocumentId,
        DocumentTitle = r.Paragraph.Document?.Title ?? "Unknown",
        ParagraphContent = r.Paragraph.Content,

        RelevanceScore = Math.Round(r.TotalScore * 100, 2),
        VectorScore = Math.Round(r.VectorScore * 100, 2),
        KeywordScore = 0,

        Rank = idx + 1,
        HighlightedWords = new List<string>(),
        ScoreBreakdown = new Dictionary<string, double>
        {
            ["vector"] = r.TotalScore
        }
    })
    .ToList();

response.TotalResults = response.Matches.Count;
response.VectorSearchTimeMs = vectorSearchTime;
response.KeywordSearchTimeMs = keywordSearchTime;

stopwatch.Stop();
_logger?.LogInformation($"Search completed in {stopwatch.ElapsedMilliseconds}ms on thread {Thread.CurrentThread.ManagedThreadId}");
response.TotalTimeMs = (int)stopwatch.ElapsedMilliseconds;

// Кэширование результата
if (request.UseCache && request.LogQuery)
{
    var cacheKey = $"search_{request.Algorithm}_{request.Query.ToLower().Trim()}";
    _cache.Set(cacheKey, response, TimeSpan.FromMinutes(30));
}

// Логирование запроса
if (request.LogQuery)
{
    await LogQueryAsync(request, response);
}

```

```

        _logger.LogInformation($"Search completed: {request.Query} → {response.Matches.Count} results in {response.TotalTimeMs}ms");

    return response;
}

private async Task InitializeAsync()
{
    _logger.LogInformation("Initializing semantic search service...");

    // Инициализация эмбедингов
    await _embeddingService.InitializeAsync();

    // Инициализация векторного хранилища
    await _vectorStore.InitializeAsync(_embeddingService.VectorDimension);

    // Загрузка лингвистических данных
    await _linguisticService.LoadDataAsync();

    // Загрузка абзацев и инициализация статистики для BM25
    var paragraphs = await _paragraphRepo.GetAllWithVectorsAsync();
    (_rankingService as RankingService)?.InitializeStats(paragraphs);

    // Предзагрузка векторов в InMemoryVectorStore
    var vectors = await _vectorRepo.GetAllWithParagraphsAsync();
    _logger.LogInformation($"Loading {vectors.Count()} vectors from DB");

    foreach (var v in vectors)
    {
        try
        {
            var floatVector = VectorMath.BytesToFloats(v.VectorData);

            if (floatVector.Length != _embeddingService.VectorDimension)
            {
                _logger.LogWarning($"Skipping vector for paragraph {v.ParagraphId}: expected {_embeddingService.VectorDimension}, got {floatVector.Length}");
                continue;
            }

            await _vectorStore.AddVectorAsync(v.ParagraphId, floatVector);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, $"Failed to load vector for paragraph {v.ParagraphId}");
        }
    }

    var pendingCount = paragraphs.Count(p => p.IndexedAt == null);
    if (pendingCount > 0)
    {
        _logger.LogInformation($"Found {pendingCount} paragraphs without vectors. Indexing...");
        await IndexPendingParagraphsAsync();
    }

    _logger.LogInformation("Semantic search service initialized");
}

```

```

public async Task<int> IndexPendingParagraphsAsync()
{
    var pending = await _paragraphRepo.GetNotIndexedAsync();
    var indexed = 0;

    _logger?.LogInformation($"Indexing {pending.Count()} pending paragraphs...");

    foreach (var paragraph in pending)
    {
        try
        {
            _logger?.LogDebug($"Generating embedding for paragraph {paragraph.Id} (length: {paragraph.Content.Length})");

            // Генерация эмбединга
            var embedding = await _embeddingService.GenerateEmbeddingAsync(paragraph.Content);

            _logger?.LogDebug($"Embedding generated: dim={embedding.Length}, first3=[{string.Join(", ", embedding.Take(3))}]");

            // Сохранение в БД
            var vectorEntity = new ParagraphVector
            {
                ParagraphId = paragraph.Id,
                VectorData = VectorMath.FloatsToBytes(embedding),
                VectorDimension = embedding.Length,
                ModelName = _embeddingService.ModelName,
                Normalized = true
            };

            await _vectorRepo.AddAsync(vectorEntity);
            await _paragraphRepo.MarkAsIndexedAsync(paragraph.Id);
            await _vectorStore.AddVectorAsync(paragraph.Id, embedding);

            paragraph.Embedding = embedding;

            indexed++;
        }
        catch (Exception ex)
        {
            _logger?.LogError(ex, $"Failed to index paragraph {paragraph.Id}");
        }
    }

    _logger?.LogInformation($"Indexed {indexed} paragraphs");
    return indexed;
}

public async Task<bool> ReindexParagraphAsync(int paragraphId)
{
    var paragraph = await _paragraphRepo.GetByIdAsync(paragraphId);
    if (paragraph == null)
        return false;

    // Удаление старого вектора
    var existingVector = await _vectorRepo.GetByParagraphIdAsync(paragraphId);

```

```

        if (existingVector != null)
        {
            _vectorRepo.Remove(existingVector);
            await _vectorStore.RemoveVectorAsync(paragraphId);
        }

        // Генерация нового
        var embedding = await _embeddingService.GenerateEmbeddingAsync(paragraph.Content);

        var vectorEntity = new ParagraphVector
        {
            ParagraphId = paragraph.Id,
            VectorData = VectorMath.FloatsToBytes(embedding),
            VectorDimension = embedding.Length,
            ModelName = _embeddingService.ModelName,
            Normalized = true
        };

        await _vectorRepo.AddAsync(vectorEntity);
        await _paragraphRepo.MarkAsIndexedAsync(paragraph.Id);
        await _vectorStore.AddVectorAsync(paragraph.Id, embedding);

        paragraph.Embedding = embedding;
        paragraph.IndexedAt = DateTime.UtcNow;

        return true;
    }

    public async Task<IndexingStatsDto> GetIndexingStatsAsync()
    {
        var total = await _paragraphRepo.CountAsync();
        var indexed = await _vectorRepo.GetIndexedCountAsync();

        return new IndexingStatsDto
        {
            TotalParagraphs = total,
            IndexedParagraphs = indexed,
            PendingParagraphs = total - indexed,
            ModelName = _embeddingService.ModelName,
            VectorDimension = _embeddingService.VectorDimension
        };
    }

    private async Task LogQueryAsync(SearchRequestDto request, SearchResponseDto response)
    {
        await Task.CompletedTask;
    }
}

```

```

SearchApiController.cs:
using Microsoft.AspNetCore.Mvc;
using SemanticSearch.Application.Interfaces;
using SemanticSearch.Core.DTO;
using SemanticSearch.Core.Enums;
using System.Text.Json.Serialization;

```



```

namespace SemanticSearch.Web.Controllers.Api
{
    [Route("api/[controller]")]
    [ApiController]
    [IgnoreAntiforgeryToken]
    public class SearchApiController : ControllerBase
    {
        private readonly ISemanticSearchService _searchService;
        private readonly ILogger<SearchApiController> _logger;

        public SearchApiController(
            ISemanticSearchService searchService,
            ILogger<SearchApiController> logger)
        {
            _searchService = searchService;
            _logger = logger;
        }

        [HttpPost("search")]
        [ProducesResponseType(typeof(SearchResponseDto), 200)]
        [ProducesResponseType(400)]
        public async Task<ActionResult<SearchResponseDto>> Search([FromBody] SearchRequestDto request)
        {
            _logger.LogInformation($"API Search: Query='{request?.Query}', Algorithm={request?.Algorithm}");

            if (string.IsNullOrEmpty(request?.Query))
            {
                _logger.LogWarning("Search request missing query");
                return BadRequest(new { error = "Query is required" });
            }

            try
            {
                var result = await _searchService.SearchAsync(request);
                return Ok(result);
            }
            catch (Exception ex)
            {
                _logger.LogError(ex, "Search API error");
                return StatusCode(500, new { error = ex.Message });
            }
        }

        [HttpGet("health")]
        public ActionResult<HealthResponse> Health()
        {
            return Ok(new HealthResponse
            {
                Status = "ok",
                Timestamp = DateTime.UtcNow
            });
        }
    }

    public class HealthResponse
    {
        public string Status { get; set; } = string.Empty;
    }
}

```

```

        public DateTime Timestamp { get; set; }
    }
}

```

HomeController.cs:

```

using Microsoft.AspNetCore.Mvc;
using SemanticSearch.Web.Models;
using System.Diagnostics;

namespace SemanticSearch.Web.Controllers
{
    public class HomeController : Controller
    {
        private readonly ILogger<HomeController> _logger;

        public HomeController(ILogger<HomeController> logger)
        {
            _logger = logger;
        }

        public IActionResult Index()
        {
            return View();
        }

        public IActionResult Privacy()
        {
            return View();
        }

        [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
        public IActionResult Error()
        {
            return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
        }
    }
}

```

SearchController.cs:

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
using SemanticSearch.Application.Interfaces;
using SemanticSearch.Core.DTO;
using SemanticSearch.Core.Enums;
using System.Diagnostics;

namespace SemanticSearch.Web.Controllers
{
    public class SearchController : Controller
    {
        private readonly ISemanticSearchService _searchService;
        private readonly ILogger<SearchController> _logger;

        public SearchController(
            ISemanticSearchService searchService,

```

```

        ILogger<SearchController> logger)
    {
        _searchService = searchService;
        _logger = logger;
    }

    /// <summary>
    /// Главная страница с формой поиска
    /// </summary>
    public IActionResult Index()
    {
        return View();
    }

    /// <summary>
    /// Обработка запроса поиска
    /// </summary>
    [HttpPost]
    [ValidateAntiForgeryToken]
    [IgnoreAntiforgeryToken]
    public async Task<IActionResult> Search(string query, SearchAlgorithm algorithm = SearchAlgorithm.Hybrid)
    {
        if (string.IsNullOrEmpty(query))
        {
            ModelState.AddModelError("", "Введите поисковый запрос");
            return View("Index");
        }

        try
        {
            _logger.LogInformation($"Searching for: '{query}' using {algorithm}");

            var request = new SearchRequestDto
            {
                Query = query.Trim(),
                Algorithm = algorithm,
                TopK = 5,
                UseCache = false,
                LogQuery = true
            };

            var response = await _searchService.SearchAsync(request);

            return View("Result", response);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Search failed");
            ViewBag.ErrorMessage = $"Ошибка поиска: {ex.Message}";
            return View("Error");
        }
    }
}

```

Index.cshtml:

@{

```

    ViewData["Title"] = "Semantic Search";
}

<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-10">
            <div class="card shadow-lg border-0">
                <div class="card-header bg-primary text-white">
                    <h3 class="mb-0">Search the neural network knowledge base</h3>
                </div>
                <div class="card-body p-4">
                    @if (ViewBag.ErrorMessage != null)
                    {
                        <div class="alert alert-danger">@ViewBag.ErrorMessage</div>
                    }

                    <form asp-controller="Search" asp-action="Search" method="post">
                        @Html.AntiForgeryToken()

                        <div class="mb-4">
                            <label for="query" class="form-label fw-bold">Your question:</label>
                            <input type="text"
                                class="form-control form-control-lg"
                                id="query"
                                name="query"
                                placeholder="Foe example: How text tokenization works?"
                                required
                                value="@ViewData["Query"]">
                        </div>

                        <div class="row align-items-end">
                            <div class="col-md-4 mb-3">
                                <label for="algorithm" class="form-label">Search type:</label>
                                <select class="form-select form-select-lg" id="algorithm" name="algorithm">
                                    <option value="Hybrid" selected>Hybrid</option>
                                    <option value="Vector">Vector</option>
                                </select>
                            </div>
                            <div class="col-md-8 mb-3">
                                <button type="submit" class="btn btn-primary btn-lg w-100">
                                    Find the answer
                                </button>
                            </div>
                        </div>
                    </form>

                    <div class="form-text text-muted mt-2">
                        <ul>
                            <li><b>Hybrid</b>: Combines semantics and exact word matches.</li>
                            <li><b>Vector</b>: Understands the meaning even if there are no matching words.</li>
                        </ul>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>

```

```

Result.cshtml:
@model SemanticSearch.Core.DTO.SearchResponseDto
@{
    ViewData["Title"] = "Search results";
}

<div class="container mt-4">
    <!-- Хлебные крошки -->
    <nav aria-label="breadcrumb">
        <ol class="breadcrumb">
            <li class="breadcrumb-item"><a asp-action="Index">Search</a></li>
            <li class="breadcrumb-item active" aria-current="page">Results</li>
        </ol>
    </nav>

    <div class="card mb-4">
        <div class="card-body bg-light">
            <h4>Search results for: "<span class="text-primary">@Model.Query</span>"</h4>
            <div class="row text-muted small">
                <div class="col">Algorithm: <span class="badge bg-secondary">@Model.AlgorithmUsed</span></div>
                <div class="col">Time: <span class="badge bg-info">@Model.TotalTimeMs ms</span></div>
                <div class="col">Founded: @Model.TotalResults paragraphs</div>
                @if (Model.FromCache)
                {
                    <div class="col text-warning">From cache</div>
                }
            </div>
        </div>
    </div>

    @if (!Model.Matches.Any())
    {
        <div class="alert alert-warning text-center p-5">
            <h5>Ничего не найдено</h5>
            <p>Try changing the query wording or using a different algorithm..</p>
        </div>
    }
    else
    {
        @foreach (var match in Model.Matches)
        {
            <div class="card mb-3 shadow-sm border-start border-primary border-4">
                <div class="card-body">
                    <div class="d-flex justify-content-between align-items-start mb-2">
                        <h5 class="card-title text-primary">@match.DocumentTitle</h5>
                        <span class="badge bg-success fs-6">@match.RelevanceScore%</span>
                    </div>

                    <p class="card-text fs-5" style="white-space: pre-wrap;">@match.ParagraphContent</p>

                    <!-- Детализация оценки (Debug info) -->
                    <div class="mt-3 pt-2 border-top">
                        <div class="row text-center mt-1 small text-muted">
                            <div class="col">
                                Vector: <b>@match.VectorScore%</b>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        }
    }
}

```

```

        </div>
    </div>
</div>
}
}
</div>

```

Program.cs:

```

using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Caching.Memory;
using SemanticSearch.Application.AdditionalClasses;
using SemanticSearch.Application.Interfaces;
using SemanticSearch.Application.Services;
using SemanticSearch.Infrastructure.Data;
using SemanticSearch.Infrastructure.Repositories;
using SemanticSearch.Infrastructure.VectorStore;
using SemanticSearch.Web.Data;
using System.Text.Json.Serialization;

namespace SemanticSearch.Web
{
    public class Program
    {
        public static async Task Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // MVC
            builder.Services.AddControllersWithViews();

            var connectionString = builder.Configuration.GetConnectionString("LocalConnection");

            // Контекст для Поиска (в Infrastructure)
            builder.Services.AddDbContext<AppDbContext>(options =>
                options.UseSqlServer(connectionString));

            // Контекст для Identity (в Web/Data)
            builder.Services.AddDbContext<ApplicationDbContext>(options =>
                options.UseSqlServer(connectionString));

            // Настройка Identity
            builder.Services.AddIdentity<IdentityUser, IdentityRole>(options =>
            {
                options.Password.RequireDigit = true;
                options.Password.RequireLowercase = true;
                options.Password.RequireUppercase = true;
                options.Password.RequireNonAlphanumeric = false;
                options.Password.RequiredLength = 6;
            })
                .AddEntityFrameworkStores<ApplicationDbContext>()
                .AddDefaultTokenProviders();

            // Memory Cache
            builder.Services.AddMemoryCache();

```

```

builder.Services.AddSingleton<ISynonymApiService, SynonymApiService>();

// Репозитории
builder.Services.AddScoped<LinguisticRepository>();
builder.Services.AddScoped<ParagraphRepository>();
builder.Services.AddScoped<VectorRepository>();

// Векторное хранилище
builder.Services.AddSingleton<IVectorStore, InMemoryVectorStore>();

// Сервисы
builder.Services.AddScoped<ILinguisticService, LinguisticService>();
builder.Services.AddScoped<ISynonymApiService, SynonymApiService>();
builder.Services.AddScoped<IRankingService, RankingService>();

// Embedding Service
var modelPath = Path.Combine(builder.Environment.ContentRootPath, "ml-models", "all-minilm-l6-v2");
builder.Services.AddSingleton<IEmbeddingService>(sp => new EmbeddingService(
    sp.GetRequiredService<ILogger<EmbeddingService>>(), modelPath));

// Главный сервис поиска
builder.Services.AddScoped<ISemanticSearchService, SemanticSearchService>();

builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter());
        options.JsonSerializerOptions.DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull;
    });
var app = builder.Build();

// Инициализация при старте (Ленивая загрузка модели)
// Модель загрузится только при первом запросе поиска, чтобы не тормозить старт

using (var scope = app.Services.CreateScope())
{
    var linguisticService = scope.ServiceProvider.GetRequiredService<ILinguisticService>();
    await linguisticService.LoadDataAsync();
}

// Middleware
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(

```

```

        name: "default",
        pattern: "{controller=Search}/{action=Index}/{id?}");

app.MapControllers();

using (var scope = app.Services.CreateScope())
{
    var embeddingService = scope.ServiceProvider.GetRequiredService<IEmbeddingService>();
    var searchService = scope.ServiceProvider.GetRequiredService<ISemanticSearchService>();
    var linguisticService = scope.ServiceProvider.GetRequiredService<ILinguisticService>();

    // лингвистические данные
    await linguisticService.LoadDataAsync();

    // _logger.LogInformation("Pre-loading embedding model...");
    await embeddingService.InitializeAsync();

    // Индексируются все абзацы
    // _logger.LogInformation("Pre-indexing paragraphs...");
    await searchService.IndexPendingParagraphsAsync();

    // _logger.LogInformation("All services pre-loaded and ready!");
}

app.Run();
}
}
}

```